

Examen Final

Data Mining Avanzado · Maestría en Explotación de Datos y Gestión del Conocimiento · Universidad Austral

15 de octubre de 2026

Ulises Catalano
Marcos Maceo
Andrés Rodríguez

Índice

Ejercicio 1 · Redes neuronales **4**
Punto I . . . 4
Punto II . . . 4
Punto III . . . 5
Punto IV . . . 5
Punto V . . . 5
Punto VI . . . 6
Punto VII . . . 7
Punto VIII . . . 7

Ejercicio 2 · Support Vector Machines **9**
Qué es y tipos de SVM . . . 9
Parámetros externos . . . 10
La característica que lo distingue . . . 10
De clasificación a regresión . . . 11

Ejercicio 3 · Statistical Learning Theory **12**
De qué trata el campo . . . 12
El indicador de complejidad con SVM . . . 12
Capacidad y dimensión VC . . . 12
La estimación heurística . . . 13

Ejercicio 4 · Algoritmos genéticos **14**
Pseudocódigo . . . 14
AG binarios y AG reales . . . 15
Implementación con software libre . . . 16

Ejercicio 5 · Componentes principales y kernels **18**
PCA en Machine Learning . . . 18
El truco del kernel . . . 19
Kernel PCA . . . 19
Cómo elegir el kernel . . . 20

Ejercicio 6 · Análisis de supervivencia **21**
Por qué supera a la regresión . . . 21
Tipos . . . 21
Implementaciones en Python y R . . . 22

Ejercicio 7 · Clustering y t-SNE **24**
Técnicas de pre-proceso . . . 24
Perfilado de los grupos obtenidos . . . 24
El algoritmo t-SNE . . . 26
Resultados sobre los dígitos . . . 27
Ventajas y limitantes . . . 27

Ejercicio 8 · AdaBoost y Gradient Boosting **28**

Diferencias entre AdaBoost y Gradient Boosting . . . 28
Cómo trabaja Gradient Boosting . . . 29

Ejercicio 9 · Random Forest **31**
El algoritmo . . . 31
Diferencias con el resto de los algoritmos de ensamblaje . . . 32
Principales usos . . . 33

Ejercicio 10 · Pipeline en Python **34**
Opción 1 . . . 34
Datos y esquema de validación . . . 34
Preprocesamiento y métricas . . . 34
Estrategia de optimización y validación cruzada . . . 35
Espacio de búsqueda de hiperparámetros . . . 35
Resultados . . . 36
Comparación de rendimiento . . . 36

Apéndice · Código **38**
A.1 LSTM serie temporal (Ejercicio 1.VI) . . . 38
A.2 Figuras CNN y celda LSTM (Ejercicio 1) . . . 41
A.3 AG binario y AG real sobre Rastrigin (Ejercicio 4) . . . 44
A.4 Alineamiento kernel-objetivo sobre Breast Cancer (Ejercicio 5) . . . 47
A.5 Atajos de regresión contra Kaplan-Meier y Cox en Rossi (Ejercicio 6) . . . 49
A.6 t-SNE y clustering sobre dígitos (Ejercicio 7) . . . 51
A.7 Pipeline Opción 1 (Ejercicio 10) . . . 56

Referencias **68**

Ejercicio 1 · Redes neuronales

Punto I

Explique el papel de las funciones de activación en redes neuronales profundas. Discuta por qué son necesarias, cómo afectan la capacidad de representación y cuáles son sus ventajas y desventajas más importantes.

La combinación agrega las entradas ponderadas, $w_0 + w_1 x_1 + \cdots + w_k x_k$, y la activación transforma esa suma en la salida de la neurona (Volpacchio, s.f.). Sin no linealidad, dos capas colapsan a $W_2(W_1 x) = (W_2 W_1)x$ y la red entera no representa más que una regresión lineal. Con activación sigmoidal, una capa oculta con suficientes neuronas aproxima cuanto se quiera cualquier función continua sobre una región cerrada y acotada (Cybenko, 1989). Con ReLU la red computa una función lineal por tramos, y la cantidad de tramos crece con la profundidad: ahí está lo que aporta apilar capas a la capacidad de representación (Goodfellow et al., 2016).

Las activaciones más comunes son el escalón, la sigmoide, tanh, softmax y ReLU, máx$(0, z)$, la más usada en redes profundas (Goodfellow et al., 2016), y sus ventajas y desventajas se reparten alrededor de dos defectos. La saturación afecta a la sigmoide, con rango $(0, 1)$, que se lee como probabilidad, y a tanh, con rango $(-1, 1)$, que está centrada en cero: en las dos, lejos del origen la derivada es casi nula y su producto capa tras capa tiende a cero (desvanecimiento del gradiente). La sigmoide arrastra además una salida que no está centrada, lo que sesga el gradiente de la capa siguiente, y las dos cuestan una exponencial. La neurona muerta es el defecto de ReLU, que no satura para $z > 0$, es una comparación en lugar de una exponencial, deja activaciones ralas (en cero exacto, no en un valor chico, así que no hay que propagarlas) y converge más rápido, pero deja de actualizarse si recibe siempre $z < 0$; Leaky ReLU y ELU lo corrigen con una pendiente chica para $z < 0$. Los dos extremos quedan fuera de esa comparación: el escalón, de salida binaria y derivada nula (salvo en 0, donde no existe), no entrena por gradiente, y softmax, que da probabilidades que suman 1, solo sirve en la salida, porque normaliza entre clases.

Punto II

Describa el proceso de entrenamiento de una red neuronal (forward pass, función de pérdida, backpropagation, optimización). Analice conceptualmente qué factores determinan la calidad del aprendizaje. Explique las diferencias esenciales entre redes "Deep" y "Shallow".

El entrenamiento busca los pesos que minimizan el error (Volpacchio, s.f.). En el forward pass cada neurona combina y activa, capa por capa, hasta la predicción o; la pérdida compara o con el objetivo t (error cuadrático, o menos el logaritmo de la probabilidad de la clase correcta). Backpropagation obtiene por regla de la cadena, desde la salida, $\delta_j = \partial E / \partial \mathrm{net}_j$, la derivada de la pérdida respecto de la entrada combinada de cada neurona. La optimización mueve cada peso contra el gradiente, $w_{ji} \leftarrow w_{ji} - \eta\, \delta_j\, o_i$ (o_i: salida que alimenta a j; $\eta \in (0, 1)$: tasa de aprendizaje), tras cada observación (online), tras cada época (batch) o tras lotes chicos.

La calidad depende de los datos frente a la cantidad de pesos, de η (chica no avanza, grande oscila), de los mínimos locales y del criterio de corte. Contra el sobreajuste, el apunte de taxonomía de la clase 4 agrupa early stopping (cortar cuando sube el error de validación), weight decay y dropout como formas de la misma idea, restringir la familia de funciones. Una red superficial (shallow) tiene una sola capa oculta. Una profunda (deep) apila capas que arman rasgos sobre rasgos (bordes, formas, objetos) y necesita menos neuronas en total que una sola capa oculta de igual expresividad;

pide más datos y el gradiente se desvanece más, lo que motivó ReLU y las conexiones residuales, que suman la entrada de un bloque a su salida (Goodfellow et al., 2016).

Punto III

Explique el concepto de "embedding" y autoencoder en el contexto de "Deep Learning". De ejemplos de uso.

Los dos están en la clase 4: `autoencoders.pdf` desarrolla el autoencoder (tipos convolucional, sparse, profundo y contractivo) y el apunte de taxonomía dedica una subsección a los word embeddings. Un embedding es un vector denso y corto que representa un objeto discreto (palabra, producto, cliente) y que la red aprende como un peso más: word2vec lo aprende para cada palabra a partir de sus vecinas, y palabras de contextos parecidos quedan próximas (Mikolov et al., 2013). Un autoencoder reconstruye su entrada: el encoder comprime a un código de menor dimensión (el cuello de botella), el decoder reconstruye y la pérdida es el error de reconstrucción, sin etiquetas. Usos: reducción de dimensión no lineal (Hinton & Salakhutdinov, 2006), anomalías (lo que se reconstruye mal) y eliminación de ruido.

Punto IV

De un tipo de red neuronal que sirve para obtener clustering. La que elija, explique brevemente cómo funciona.

Se elige el SOM (Self Organizing Map, mapa autoorganizado) del apunte; k-means y DBSCAN no son redes, y el competitive learning (neuronas que compiten sin relación de vecindad) agrupa pero no ordena el mapa (Kohonen, 1990; Volpacchio, s.f.). Cada neurona U_j de la grilla tiene pesos w_j de la dimensión de la entrada. Para cada x, la ganadora U_{win} minimiza $\|x - w_j\|$ y se actualiza $w_j(t) = w_j(t-1) + (x - w_j)\,\eta(t)\,N_{win}(j,t)$: el entorno N_{win} se achica hasta la ganadora y la tasa $\eta(t)$ achica el paso. Cada neurona queda como prototipo, cada observación va a su ganadora y las neuronas vecinas parecidas forman un cluster.

Punto V

¿Qué es una convolución y como aplica en las redes neuronales? De un diagrama de las distintas capas de una red convolucional y explique.

Una convolución recorre un filtro, una matriz de pesos mucho más chica que la entrada, sobre un mapa de píxeles; cada suma de productos forma un punto del mapa de características. El *kernel* de las diapositivas de CNN es ese filtro, no el de SVM. Con lado n, filtro f, relleno del borde (padding) r y salto (stride) s, la salida mide $\lfloor (n + 2r - f)/s \rfloor + 1$ (`deep_learning_CONV_Ng.pptx`; Volpacchio, s.f.). LeCun et al. (1998) aprenden el filtro por backpropagation, con conexión local y pesos compartidos: menos parámetros que una capa densa, y un rasgo desplazado produce la misma respuesta desplazada (equivariancia a la traslación). El pooling por el máximo de cada ventana agrega invariancia a corrimientos chicos. En la Figura 1, el aplanado convierte los mapas en un vector, la capa densa combina los rasgos y softmax da la probabilidad de cada clase.

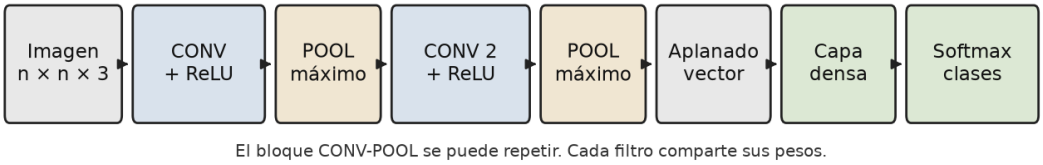


Figura 1: Capas de una CNN típica de clasificación de imágenes.

Nota. Elaboración propia a partir de `deep_learning_CONV_Ng.pptx` (Volpacchio, s.f.).

Punto VI

¿Qué es una red neuronal recurrente clásica? ¿Las redes LSTM en que se diferencian de las redes recurrentes clásicas? De un diagrama de su topología y explíquela brevemente. Cantidad de parámetros que utiliza en base a su configuración. Genere un ejemplo completo en Python de aplicación de una red LSTM para predecir una serie de tiempo arbitraria. Pruebe diferentes configuraciones de capas y cantidad de neuronas.

Una RNN (red recurrente) clásica, las Recurrent Networks del apunte (Volpacchio, s.f.), reutiliza su estado, $h_t = f(Wx_t + Uh_{t-1} + b)$, con los mismos pesos en cada paso; hacia atrás el gradiente se multiplica por U una vez por paso y en dependencias largas se desvanece o explota. La LSTM (Long Short-Term Memory), que el apunte `Dimensiones y Arquitectura LSTM` de la clase 4 desarrolla puerta por puerta, agrega una celda c_t que se actualiza con una suma de lo nuevo y lo anterior (Hochreiter & Schmidhuber, 1997) y una puerta de olvido (Gers et al., 2000). En la Figura 2 (s: sigmoidal; $\odot$: producto elemento a elemento), cada puerta es $g_t = s(W_g x_t + U_g h_{t-1} + b_g)$, $g \in \{f, i, o\}$: f_t dosifica cuánto de c_{t-1} se conserva, i_t cuánto de la candidata $\tilde{c}_t$ (tanh) se escribe y o_t cuánto sale en h_t; con $f_t \approx 1$ el gradiente cruza la celda.

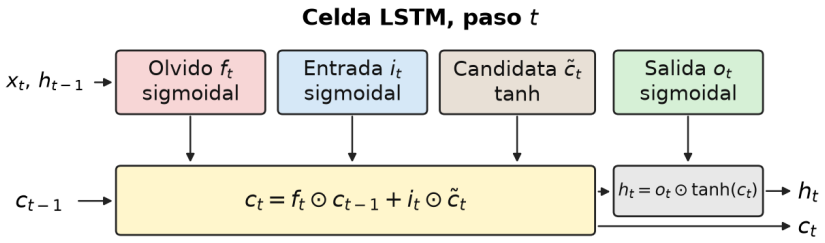


Figura 2: Celda LSTM en un paso temporal.

Nota. Elaboración propia. x_t y h_{t-1} alimentan las cuatro cajas superiores. La celda se repite en cada paso temporal con los mismos pesos, y h_t y c_t pasan al paso siguiente; en las configuraciones de dos capas, una segunda LSTM recorre la secuencia de salidas de la primera.

Con la fórmula de ese apunte, $4 \times [(\text{units} \times \text{input_dim}) + \text{units}^2 + \text{units}]$ por capa (tres puertas y la candidata, cada una con W, U y sesgo), `lstm_32` tiene $4 \times [(32 \times 1) + 32^2 + 32] = 4352$ parámetros, los que `model.summary()` asigna a la capa LSTM; los 4385 del total agregan la densa de salida. El script del Apéndice usa dos senos más ruido gaussiano de desvío 0,1, ventanas de 20 valores, 20 épocas con Adam (paso adaptado a cada peso), corte temporal 80/20 sin mezclar, el último 10 % del entrenamiento como validación y tres semillas, porque con una el orden entre configuraciones podía ser azar.

Tabla 1: LSTM: parámetros y error (media de tres semillas).

Config	Neuronas por capa	Fórmula	Total Keras	MAE val.	MAE test
lstm_32	32	4352	4385	0,1051	0,0954 ± 0,0028
lstm_64	64	16896	16961	0,1059	0,0945 ± 0,0007
lstm_32_16	32-16	7488	7505	0,1152	0,0974 ± 0,0009
lstm_64_32	64-32	29312	29345	0,1115	0,0966 ± 0,0018

Nota. Elaboración propia. Fórmula: solo capas LSTM. MAE test: media ± DE entre semillas.

Gana por validación lstm_32 (0,1051); su MAE de test, 0,0954, mejora un 9 % a la persistencia (predecir con el valor anterior, 0,1052) y queda un 28 % sobre el piso de ruido (señal limpia contra observada, 0,0746). Dos capas validan peor en todas las corridas (peor con una: 0,1067; mejor con dos: 0,1094). En test `lstm_64` gana por menos de un desvío; se mantiene la elección, porque decidir con el test le quita valor de estimación.

La serie es sintética, y eso acota lo que el ensayo puede mostrar. A favor: con dos senos y ruido de desvío conocido hay dos referencias contra las cuales medir, la persistencia y el piso de ruido, que en una serie real no se conocen. En contra, y pesa más: las cuatro configuraciones quedan dentro de su propia dispersión, así que el experimento no alcanza para ordenarlas y la comparación de arquitecturas que pide el enunciado queda sin respuesta firme. Separarlas pediría una serie real, con tendencia y estacionalidad, donde la memoria larga de la LSTM tenga algo que aportar frente a un modelo lineal.

Punto VII

Describa el algoritmo encoder-decoder. Usos habituales.

Una red recurrente sola exige entrada y salida del mismo largo, y traducir no funciona así. El encoder-decoder acopla dos redes: el encoder resume la entrada en un vector de contexto y el decoder genera la salida de a un elemento, realimentado con lo ya generado, hasta el símbolo de fin (Sutskever et al., 2014). Así se usa una vez entrenado; durante el entrenamiento el decoder recibe en cada paso el objetivo verdadero y no lo que predijo (*teacher forcing*), y en la generación se puede tomar el elemento más probable de cada paso o explorar varias secuencias parciales a la vez (*beam search*). El cuello de botella es ese vector de tamaño fijo, que carga con la oración entera y degrada el rendimiento cuanto más larga es; la atención deja al decoder consultar en cada paso todos los estados del encoder (Bahdanau et al., 2015), y el Transformer descarta la recurrencia y conserva solo la atención (Vaswani et al., 2017). Usos: traducción, resumen, voz y descripción de imágenes (encoder convolucional, decoder recurrente); el autoencoder del Punto III es el caso en que la salida pedida es la entrada.

Punto VIII

Explique de qué manera la arquitectura de una red neuronal artificial influye en su capacidad de representación y generalización.

Describa qué se entiende por capacidad de representación y por capacidad de generalización en el contexto del aprendizaje automático.

- *Analice cómo distintos elementos arquitectónicos (cantidad de capas, número de neuronas, funciones de activación, regularización, tipo de conexión, etc.) modifican estas capacidades.*

- *Discuta el rol del sesgo y la varianza en redes neuronales y cómo la arquitectura puede afectar este equilibrio.*
- *Incluya ejemplos conceptuales (no numéricos) que ilustren cuándo una red puede estar subajustada (underfitting) o sobreajustada (overfitting) por decisiones arquitectónicas.*
- *Fundamente cómo un cambio arquitectónico puede mejorar o empeorar el desempeño, incluso sin modificar los datos de entrenamiento.*
- *Reflexione sobre por qué modelos con más parámetros no siempre son mejores, y cómo esto se conecta con el principio de mínima complejidad y el aprendizaje profundo moderno.*

La capacidad de representación es el conjunto de funciones que la arquitectura computa al variar sus pesos; la de generalización, sostener el desempeño en datos no vistos (brecha entre error de entrenamiento y de validación; Volpacchio, s.f.). Más capas y neuronas agrandan la primera si la activación es no lineal; weight decay, early stopping y dropout (apagar neuronas al azar; Srivastava et al., 2014) la restringen. La convolución supone rasgos locales repetidos, la recurrencia orden temporal y la capa densa nada: ese supuesto es el que mejora la generalización cuando se cumple, porque restringe la familia sin pagar error de ajuste, y el que la arruina cuando no; las conexiones residuales permiten entrenar más de cien capas (He et al., 2016).

Sesgo es la distancia entre la predicción promedio sobre las muestras posibles y el valor verdadero; varianza, cuánto cambia el ajuste de una muestra a otra; el error esperado suma sesgo al cuadrado, varianza y ruido. Un perceptrón sin capa oculta no separa el XOR (clases en esquinas opuestas de un cuadrado) y subajusta: sesgo alto. Un MLP (perceptrón multicapa) muy ancho, sin regularización y con pocos datos, memoriza el ruido y sobreajusta: varianza alta. Con los mismos datos, una capa oculta mejora al primero, porque agranda la familia de funciones hasta incluir la frontera que el XOR necesita, y más neuronas empeoran al segundo, porque la agrandan más allá de lo que esos datos pueden identificar y el ajuste pasa a seguir el ruido.

Más parámetros no garantizan más acierto: la LSTM 64-32 tiene 7 veces los parámetros de `lstm_32` y valida peor. La mínima complejidad pide la familia más simple compatible con los datos: la regla $d/l < 0{,}05$ del Ejercicio 3, con $d = k + 1$ exacto para un hiperplano. Con d aproximada por los pesos, `lstm_32` da $d/l \approx 5{,}2$ y aun así mejora a la persistencia: contar pesos sobrestima la capacidad de una red, sin invalidar la regla lineal. Una explicación posible es el doble descenso (Belkin et al., 2019): pasado el ajuste perfecto, el error de test vuelve a bajar porque el descenso de gradiente estocástico (SGD) y la regularización limitan la complejidad usada. Es una hipótesis: no se midió el error de entrenamiento.

Fuentes del Ejercicio 1. Material de cátedra (Volpacchio, s.f.): clase 3 (redes, MLP, SOM; `deep_learning_CONV_Ng.pptx`) y clase 4 (`autoencoders.pdf`, `Apunte - Dimensiones y Arquitectura LSTM`, `resumen_redes_recurrentes_2024_`, `atencion_mecanismo` y taxonomía). Código: A.1 y A.2.

Ejercicio 2 · Support Vector Machines

Mencione y describa los tipos de SVM. Que parámetros externos necesita el algoritmo en su versión no lineal separable. ¿Qué característica especial tiene SVM en relación al resto de los algoritmos de Machine Learning?

¿Cómo se adapta SVM clasificación, para poder usarlo como regresión?

Qué es y tipos de SVM

Una máquina de vectores soporte (SVM, por Support Vector Machine) es un algoritmo de clasificación que busca un hiperplano separador entre dos clases (Volpacchio, s.f.). En las diapositivas de la materia el título es Vector Support Machines (VSM); se unifica a SVM, la sigla de Burges (1998), la referencia de base del apunte.

Un hiperplano es la generalización de una recta a espacios de mayor dimensión: en un espacio de k variables explicativas queda definido por la ecuación $w \cdot x - b = 0$, donde w es un vector de pesos y b un escalar. Entre todos los hiperplanos que separan las clases, SVM elige el de margen máximo: el margen es la distancia entre los dos hiperplanos paralelos H_{+1} y H_{-1} que pasan por las observaciones más cercanas de cada clase, y vale $2/\|w\|$.

La maximización del margen equivale a la minimización de $\|w\|$ con la restricción $y_i(w \cdot x_i - b) \geq 1$ para todos los puntos. El problema resultante es de programación cuadrática, una optimización con función objetivo cuadrática y restricciones lineales, que posee un mínimo global dentro de un conjunto convexo de soluciones (Burges, 1998). Los puntos de entrenamiento que quedan sobre H_{+1} o H_{-1}, los más cercanos al hiperplano clasificador, se llaman vectores soporte y son los únicos que definen la solución. El problema tiene una versión dual, una reformulación equivalente que, por medio de multiplicadores de Lagrange (un peso por cada restricción), deja de buscar w directamente y pasa a ponderar cada punto de la muestra: cada punto recibe un multiplicador $\alpha_i \geq 0$, solo los vectores soporte terminan con $\alpha_i > 0$ y el resto de la muestra no participa del cálculo del hiperplano, porque $w = \sum_i \alpha_i y_i x_i$.

El apunte presenta cuatro versiones; cada una relaja un supuesto de la anterior, y por eso aparece en cada paso un componente nuevo del algoritmo (Volpacchio, s.f.).

- SVM lineal con datos separables (margen duro). Es la versión básica descrita arriba: exige que ningún punto quede entre H_{+1} y H_{-1}, por lo que solo funciona si las clases se separan en forma perfecta con un hiperplano.
- SVM lineal con separación imperfecta (margen blando). Admite ruido en la frontera: se introducen las variables de holgura $\xi_i \geq 0$ (slack), que miden cuánto viola cada punto la frontera de su clase, y se agrega a la función objetivo el término de penalización $C \sum_i \xi_i$. El parámetro C regula el compromiso entre un margen ancho y pocos errores de clasificación (Cortes & Vapnik, 1995).
- SVM no lineal. Cuando la frontera entre clases no es lineal, los datos se transforman con una función ϕ desde el Input Space (espacio de entrada, donde viven las variables originales) hacia el Feature Space (espacio de características de mayor dimensión), donde el problema tiene alta probabilidad de volverse linealmente separable.

  El costo de esa transformación se evita con el truco del kernel: en la formulación dual los datos solo aparecen como productos internos, y una función kernel $K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$

calcula ese producto sin pasar por el Feature Space. La frontera resultante es lineal en el espacio transformado y no lineal en el original.

- SVM para regresión (SVR, Support Vector Regression). Adapta el esquema anterior a la predicción de una variable continua; se desarrolla al final de este ejercicio.

El apunte se detiene en esas cuatro versiones. Fuera de él existen otras: ν-SVM reemplaza C por $\nu \in (0, 1]$, una cota de la fracción de errores y de vectores soporte (Schölkopf et al., 2000); One-Class SVM separa una sola clase del resto del espacio para detectar anomalías (Schölkopf et al., 2001); y con más de dos clases se combinan clasificadores binarios, uno contra uno o uno contra el resto (Hsu & Lin, 2002).

Parámetros externos

En la versión no lineal separable los datos, una vez llevados al Feature Space, se separan sin error. Los parámetros externos (los valores que el usuario fija antes del entrenamiento y que el algoritmo no aprende de los datos) son entonces solo el kernel y sus parámetros; C no interviene, porque sin holgura no hay errores que penalizar, lo que equivale a $C \to \infty$. Como en la práctica nunca se sabe de antemano si la separación será perfecta, también se fija C, y la lista queda así.

- La elección de la función kernel: polinómica, de base radial o sigmoide, entre las más usadas.
- Los parámetros propios del kernel elegido: el grado q en el kernel polinómico $K(x, x_i) = \langle x, x_i \rangle^q$; la dispersión σ en el kernel de función de base radial (RBF), $K(x, x_i) = \exp(-\|x - x_i\|^2/2\sigma^2)$, que en scikit-learn se escribe con $\gamma = 1/(2\sigma^2)$ (un γ grande es un kernel angosto); los parámetros κ y θ en el kernel sigmoide, $K(x, x_i) = \tanh(\kappa\langle x, x_i \rangle - \theta)$, que no satisface la condición de Mercer para toda combinación de valores. La condición de Mercer es el requisito que garantiza que una función simétrica se comporte como un producto interno: la matriz de Gram G, con los valores $K(x_i, x_j)$ para todos los pares de datos, debe ser semidefinida positiva ($z^\top G z \geq 0$ para todo vector z).
- Si se admite margen blando, la constante de penalización C, que en el dual acota los multiplicadores: $C \geq \alpha_i \geq 0$.

Estos valores se eligen por prueba sobre datos de validación. La presentación de la materia lo ejemplifica con una herramienta en Visual Basic: corridas con distintos valores de C, con kernel RBF y polinómicos de varios grados, y con distintos tamaños de conjunto de entrenamiento, con la eficacia de clasificación como criterio de comparación (Volpacchio, s.f.).

La característica que lo distingue

La característica que distingue a SVM es que el control de la capacidad está dentro del propio criterio que optimiza y tiene respaldo en la teoría del Ejercicio 3. En otros algoritmos de la materia la regularización se agrega aparte y se ajusta por validación (weight decay en redes, tamaño de hoja en árboles); SVM sigue el paradigma de minimización del riesgo estructural (Vapnik, 1995): maximizar el margen controla a la vez el error de entrenamiento y la capacidad del modelo.

Además, la solución depende solo de los vectores soporte, unos pocos puntos de la muestra. Vapnik (1995) da por eso una cota del riesgo real basada en el procedimiento leave-one-out (dejar uno afuera: entrenar con todos los datos menos uno, evaluar sobre el punto excluido y repetir para cada punto):

$$E[P(\text{error})] \leq \frac{E[\#\{\text{vectores soporte}\}]}{n}$$

donde n es la cantidad de observaciones de entrenamiento y las esperanzas son sobre las muestras posibles. Con una sola corrida, la proporción de vectores soporte es una aproximación de esa cota, obtenida sin armar un conjunto de prueba aparte (hold-out). Otros algoritmos estiman el error sin hold-out, como el error out-of-bag de Random Forest, pero no dan una cota leída de la propia solución. El problema de optimización, además, es convexo (mínimo global), a diferencia de las redes neuronales, que pueden quedar atrapadas en mínimos locales.

De clasificación a regresión

Para usar SVM como regresión se cambia el criterio de ajuste: en lugar del margen entre clases se mide el error de aproximación entre la predicción $f(x) = w \cdot x - b$ y el valor observado y. La pieza nueva es la función de pérdida lineal de Vapnik con zona de ε-insensibilidad (Vapnik, 1995):

$$E_\varepsilon(y, f(x)) = \begin{cases} 0, & \text{si } |y - f(x)| \le \varepsilon, \\ |y - f(x)| - \varepsilon, & \text{si } |y - f(x)| > \varepsilon. \end{cases}$$

Esta pérdida define un tubo de radio ε alrededor de la función: los puntos dentro del tubo no aportan error, y los que quedan fuera aportan solo el excedente sobre ε. El procedimiento del apunte tiene tres pasos: se calcula el riesgo empírico con la pérdida de ε-insensibilidad, se controla la capacidad con la minimización de $\frac{1}{2}\|w\|^2$, el mismo término que en clasificación, y se combinan ambos aspectos en el funcional $R = \frac{1}{2}\|w\|^2 + C\sum_i(\xi_i + \xi_i^*)$, donde las variables de holgura ξ_i y ξ_i^* miden cuánto se sale cada punto del tubo por arriba y por abajo.

El problema dual resultante es análogo al de clasificación y admite el truco del kernel para regresiones no lineales. Cada punto tiene ahora dos multiplicadores, α_i y α_i^*, uno por cada borde del tubo, y la función de aproximación final queda $F(x) = \sum_i(\alpha_i - \alpha_i^*)K(x_i, x) - b$, donde los vectores soporte son los puntos que caen sobre el borde del tubo o fuera de él. El parámetro externo adicional respecto de la clasificación es ε, el radio del tubo, que fija cuánta desviación se tolera sin costo.

Fuentes del Ejercicio 2. Material de cátedra (Volpacchio, s.f.): presentación `SVM Universidad Austral.pdf` (clase 2, que reproduce las diapositivas de SLT de la clase 1), en particular las secciones de la versión lineal, la separación imperfecta, el kernel trick, la cota por vectores soporte y Support Vector Regression. Burges (1998) para la formulación del clasificador; Cortes y Vapnik (1995) para el margen blando; Vapnik (1995) para la cota por vectores soporte y la pérdida con zona de ε-insensibilidad; Schölkopf et al. (2000, 2001) y Hsu y Lin (2002) para las variantes fuera del apunte.

Ejercicio 3 · Statistical Learning Theory

Explique brevemente de que trata el campo de Statistical Learning Theory. ¿Qué indicador muestra la complejidad de un data set usando el algoritmo SVM? ¿Qué es la capacidad de un algoritmo y qué manera heurística propone Vapnik-Chervonenkis para estimarla, dado un data set de 1000 observaciones con k variables (deje de lado el caso particular de SVM en el cual existe una forma matemática de estimarla)?

De qué trata el campo

La Statistical Learning Theory (SLT, teoría del aprendizaje estadístico) es el marco teórico del aprendizaje automático: estudia cómo inducir, a partir de la información empírica de una muestra, el comportamiento de una población, con un riesgo de error medible en términos de probabilidad. El planteo formal parte de un conjunto de entrenamiento de pares (x_i, y_i) generados por una distribución de probabilidad P desconocida, y de una función de pérdida que cuantifica el costo de cada error de predicción. Con esos elementos se definen dos riesgos: el riesgo empírico $R_{emp}(f)$, el promedio de la pérdida sobre la muestra de entrenamiento, y el riesgo real $R(f)$, la esperanza de la pérdida sobre la distribución completa, que es el que importa y no se puede calcular porque P es desconocida (Volpacchio, s.f.).

El problema central del campo es que un riesgo empírico pequeño no implica un riesgo real pequeño. El apunte lo ilustra con un clasificador que memoriza: responde la etiqueta guardada si el punto ya apareció en el entrenamiento y una etiqueta fija en caso contrario. Ese clasificador tiene riesgo empírico cero y sin embargo no aprende nada. La teoría de Vapnik y Chervonenkis establece bajo qué condiciones la minimización del riesgo empírico es consistente, es decir, converge al mejor clasificador dentro del espacio de funciones del algoritmo a medida que crece la muestra: la condición es que la capacidad de ese espacio esté restringida. De este marco surgió el desarrollo de SVM.

El indicador de complejidad con SVM

Con el algoritmo SVM, el indicador de la complejidad de un conjunto de datos es la cantidad de vectores soporte de la solución, los puntos definidos en el Ejercicio 2 que determinan el hiperplano. Clases bien separadas se resuelven con pocos vectores soporte; una frontera con clases superpuestas obliga a retener muchos puntos. Por la cota del Ejercicio 2, una proporción alta de vectores soporte anticipa un error de generalización alto: el problema es complejo para ese algoritmo.

Capacidad y dimensión VC

La capacidad de un algoritmo es la aptitud de su familia de funciones para adaptarse a los datos: cuanto más grande es el espacio de funciones entre las que el algoritmo puede elegir, más patrones distintos puede representar y más riesgo corre de sobreajuste, la memorización de la muestra con pérdida de generalización. La medida formal de la capacidad es la dimensión VC (Vapnik-Chervonenkis), definida a partir del concepto de shattering: un conjunto de puntos queda shattered (discriminado por completo) por una familia de funciones si, para cualquier asignación de etiquetas a esos puntos, existe una función de la familia que los clasifica sin error. La dimensión VC es la cantidad máxima de puntos que la familia puede shatter. El ejemplo del apunte: las rectas orientadas en el plano (rectas con un lado positivo y otro negativo) pueden shatter tres puntos pero no cuatro, por lo que su dimensión VC es 3; en general, los hiperplanos orientados en un espacio de k dimensiones tienen dimensión VC $k + 1$.

La dimensión VC importa porque aparece en la cota de riesgo de Vapnik y Chervonenkis, que relaciona el riesgo real con el empírico para la función elegida, indexada por α:

$$R(\alpha) \leq R_{\text{emp}}(\alpha) + \underbrace{\sqrt{\frac{d\left(\ln(2l/d)+1\right) - \ln(\eta/4)}{l}}}_{\text{intervalo de confianza VC}},$$

que vale con probabilidad $1-\eta$ (η es aquí la probabilidad de que la cota falle, no la tasa de aprendizaje del Ejercicio 1). El intervalo crece con la dimensión VC d y decrece con el tamaño de muestra l. En las diapositivas esa dimensión aparece como h dentro de la cota y como d en la regla d/l; aquí se unifica en d. La lectura operativa es la minimización del riesgo estructural: no alcanza un R_{emp} bajo; hay que elegir la familia cuya suma de riesgo empírico e intervalo de confianza VC sea la menor (Vapnik, 1995; Volpacchio, s.f.).

La estimación heurística

Fuera del caso de SVM, la dimensión VC en general no se calcula en forma exacta. La manera heurística que propone la teoría, según el apunte, es una regla sobre la relación entre capacidad y datos: $d/l < 0{,}05$, para que el intervalo de confianza VC sea chico frente al riesgo empírico (Volpacchio, s.f.). La estimación de d cuenta los parámetros libres del modelo; para un clasificador del tipo hiperplano sobre k variables, $d = k+1$, el valor exacto del párrafo anterior. Esa vía es la genérica, no la fórmula de capacidad propia de SVM, donde el margen acota la dimensión VC y que acá queda fuera. El propio apunte avisa que contar parámetros no equivale a medir la capacidad y remite al contraejemplo de Vapnik (1995); de ahí que la vía sea heurística. Para modelos no lineales contar parámetros es solo una aproximación, y el Ejercicio 1.VIII muestra que en una red sobrestima la capacidad.

Con $l = 1000$ observaciones y k variables, la regla exige $d < 0{,}05 \times 1000 = 50$; con $d = k+1$, se cumple con hasta 48 variables. El apunte agrega otra regla para el control del sobreajuste: 30 veces más datos que parámetros si hay ruido y 5 veces si no lo hay. Con 1000 observaciones eso da hasta 33 parámetros con ruido ($k \leq 32$) y hasta 200 sin ruido ($k \leq 199$). Se adopta la regla de 30 veces para datos reales, que casi siempre tienen ruido, y $d/l < 0{,}05$ como la heurística de Vapnik-Chervonenkis. Si hay más variables, las salidas son reducir la dimensión, por ejemplo con componentes principales (combinaciones lineales de las variables que concentran su varianza), o conseguir más observaciones.

La regla es una condición suficiente, no necesaria. En el Ejercicio 10, Breast Cancer tiene $k = 30$ variables y 455 casos de entrenamiento: un clasificador lineal tendría $d/l = 0{,}068$, por encima de 0,05, y aun así los cuatro modelos tienen un F1 de al menos 0,95 en el hold-out. Se interpreta que la regla es conservadora: cuando se cumple, el intervalo de confianza VC es chico; cuando no se cumple, no implica que el modelo sobreajuste.

Fuentes del Ejercicio 3. Material de cátedra (Volpacchio, s.f.): presentación `SLT Universidad Austral1.pdf` y diapositivas `Statistical Learning Theory.pptx` (clase 1), en particular las secciones de riesgo empírico y real, consistencia, dimensión VC, intervalo de confianza VC y minimización del riesgo estructural. Vapnik (1995) para la formulación general de la teoría y la cota de riesgo.

Ejercicio 4 · Algoritmos genéticos

Escriba el pseudocódigo de algoritmos genéticos. Explique brevemente las principales diferencias entre AG binarios y reales. De 3 casos de aplicación de AG y sus respectivas fuentes (paper, autor, fecha), con breve explicación. Cómo lo implementaría usando programas disponibles open-source (cita y breve explicación).

Pseudocódigo

Un algoritmo genético (AG) es un procedimiento probabilístico de optimización que mantiene una población de soluciones candidatas y la hace evolucionar con mecanismos inspirados en la selección natural (Goldberg, 1989). Cada candidata se codifica en un cromosoma, una cadena de genes (posiciones con un valor) que representa una posible solución, y se evalúa con una función de aptitud o *fitness*, que mide qué tan buena es esa solución para el problema. En cada generación se seleccionan cromosomas, con mayor probabilidad los más aptos, y se los altera con dos operadores genéticos: el cruce, que intercambia segmentos entre dos padres, y la mutación, que altera genes individuales al azar. El interés del método está en que no requiere gradiente ni convexidad de la función objetivo, y en que combina la explotación de las mejores soluciones con la exploración del resto del espacio de búsqueda (Volpacchio, s.f.).

El pseudocódigo sigue la estructura de *evolution program* del material de clase, tomada de Michalewicz (1996):

```
Inicio
    t <- 0
    Inicializar P(t)          # poblacion inicial aleatoria de pop_size
     ↪ cromosomas
    Evaluar P(t)              # fitness de cada cromosoma
    Mientras no se cumpla la condicion de terminacion:
        t <- t + 1
        Seleccionar P(t) a partir de P(t-1)  # ruleta proporcional al fitness o
         ↪ torneo
        Alterar P(t)          # cruce con probabilidad p_c; mutacion con
         ↪ probabilidad p_m
        Evaluar P(t)          # P(t) reemplaza por completo a P(t-1)
        # elitismo opcional: copiar el mejor de P(t-1) hacia P(t)
    Fin mientras
    Devolver el mejor cromosoma encontrado
Fin
```

La selección clásica es la ruleta: a cada cromosoma v_i se le asigna una probabilidad $p_i = \text{eval}(v_i)/F$, donde F es el fitness total de la población, y la nueva generación se sortea con esa distribución. La alternativa del torneo toma al azar unos pocos cromosomas y se queda con el mejor. Los parámetros externos son el tamaño `pop_size`, la probabilidad de cruce p_c, la de mutación p_m y el criterio de corte: un número fijo de generaciones o el estancamiento del mejor fitness. El elitismo copia al mejor individuo de $P(t-1)$ hacia $P(t)$ para que el azar no lo elimine; las diapositivas del ciclo básico no lo exigen.

AG binarios y AG reales

En el AG binario cada cromosoma es una cadena de bits. Para una variable continua en un dominio $[a, b]$, el dominio se discretiza: con m bits se obtienen 2^m valores posibles y la decodificación es $x = a + \text{decimal}(\text{cadena}_2) \cdot (b-a)/(2^m - 1)$. La precisión queda atada al largo de la cadena; en el ejemplo del material de clase, el intervalo $[-1, 2]$ con seis decimales exige 22 bits. Los operadores son combinatorios: cruce de un punto, que corta ambas cadenas en una posición aleatoria e intercambia las colas, y mutación bit a bit. El teorema del esquema de Holland (1975) queda anclado a esta representación. Un esquema es un molde de cadenas armado con 0, 1 y el símbolo $*$ de indiferencia; los esquemas cortos, de pocas posiciones fijas y con fitness superior al promedio reciben un número exponencialmente creciente de representantes, porque el cruce y la mutación rara vez los rompen.

En el AG real el cromosoma es directamente un vector de números reales, sin decodificación. El cruce pasa a ser una operación numérica sobre los padres: el cruce aritmético toma $\lambda p_1 + (1-\lambda)p_2$ con $\lambda \in [0, 1]$ (Michalewicz, 1996), y el cruce de mezcla BLX-α sortea cada gen del hijo en el intervalo que forman los dos padres, ampliado en una fracción α de su ancho. La mutación suma una perturbación gaussiana al gen. La Tabla 2 resume las diferencias.

Tabla 2: Diferencias entre el AG binario y el AG real.

Aspecto	AG binario	AG real
Representación	Cadena de bits	Vector de reales
Decodificación	Necesaria para variables continuas	No hace falta
Cruce	De uno o varios puntos	Aritmético, de mezcla (BLX-α)
Mutación	Cambio de bit (0 a 1 o 1 a 0)	Perturbación gaussiana
Precisión	Fijada por la cantidad de bits	La de la máquina
Uso natural	Problemas discretos, selección de variables	Optimización continua

Nota. Elaboración propia a partir de Volpacchio (s.f.) y Michalewicz (1996).

Para problemas continuos se prefiere la codificación real, porque una precisión alta en binario exige cromosomas largos y porque valores vecinos del dominio pueden diferir en muchos bits, lo que entorpece el ajuste fino. La binaria se reserva para espacios discretos, donde es natural: en selección de variables cada bit indica si una variable entra al modelo, como en el notebook de la clase 7 (Volpacchio, s.f.). La programación genética, donde el individuo es un programa representado como árbol, es una derivación del mismo esquema y no se toma como el pseudocódigo pedido.

Para ver la diferencia en un caso concreto se programaron los dos AG con DEAP sobre la función de Rastrigin en dos variables, que es el banco de pruebas del notebook de algoritmos evolutivos de la clase 7, de donde se toman también el dominio, el cruce de mezcla con $\alpha = 0{,}5$, la mutación gaussiana de desvío 0,3 y el torneo de tres, $f(x) = 20 + \sum_{i=1}^{2} \left(x_i^2 - 10\cos 2\pi x_i\right)$ en $[-5{,}12;\ 5{,}12]^2$, cuyo mínimo global $f = 0$ está en el origen, rodeado de una grilla de mínimos locales cerca de los puntos enteros. Se descartó la esfera $x_1^2 + x_2^2$ del primer borrador, porque cualquier método de gradiente la resuelve y no distingue las codificaciones. Los dos AG comparten población de 50, 100 generaciones, torneo de tres, $p_c = 0{,}8$ y probabilidad de mutar un individuo de 0,2. El binario usa 16 bits por variable, cruce de un punto y mutación con probabilidad 1/32 por bit; el real usa BLX-0,5 y mutación gaussiana con desvío 0,3. Se corrieron 20 semillas por codificación (Tabla 3).

Tabla 3: AG binario y AG real sobre Rastrigin: estado final de las corridas.

Codificación	f < 0,01	Cuenca global	Valle vecino	Mediana del mejor f	Generación mediana a f < 0,01
Binaria	9	10	10	0,587	19
Real	10	15	5	0,021	13

Nota. Elaboración propia. Conteos sobre 20 corridas por codificación. Cuenca global: $|x_1|$ y $|x_2|$ menores que 0,5; valle vecino: alguna coordenada a menos de 0,1 de ± 1. La generación mediana se calcula sobre las corridas que bajaron de 0,01.

Ninguna codificación garantiza el óptimo: las dos llegan a $f < 0{,}01$ en cerca de la mitad de las corridas.

La diferencia está en cómo fallan. Las 10 binarias que no entran a la cuenca global quedan clavadas en un valle vecino, con alguna coordenada en ± 1: el entero que codifica $x = 1$ difiere en 6 bits del más cercano de los dos enteros que rodean a $x = 0$, y con probabilidad $1/32$ por bit una mutación que cambie esos bits a la vez es casi imposible. Es el fenómeno anticipado al comparar las codificaciones: valores vecinos del dominio con códigos lejanos. Las reales entran a la cuenca global en 15 de 20 corridas; 5 de ellas quedan a menos de 0,07 del origen sin terminar el ajuste fino, porque un desvío de 0,3 es grueso a esa escala.

Se interpreta que, con el mismo presupuesto de 5.050 evaluaciones, la codificación real explora mejor la vecindad del óptimo; la binaria necesitaría una mutación más agresiva o un código Gray, en el que enteros consecutivos difieren en un solo bit (Goldberg, 1989).

Tres casos de aplicación documentados, en dominios distintos:

1. **Programación de tareas (*job shop scheduling*).** Davis (1985) aplicó AG a la asignación de trabajos a máquinas en una planta: el cromosoma codifica preferencias de orden entre tareas y el fitness es el tiempo total del cronograma. Es uno de los primeros usos industriales del método.
2. **Entrenamiento de redes neuronales.** Montana y Davis (1989) ajustaron los pesos de una red hacia adelante (*feedforward*) con un AG de codificación real sobre datos de sonar, como alternativa a la retropropagación cuando la superficie de error tiene muchos mínimos locales.
3. **Selección de variables en datos de alta dimensión.** Feng (2024) usa un AG en el que cada individuo representa un subconjunto de variables y el fitness es el desempeño del clasificador, y lo evalúa sobre Fashion-MNIST y sobre ocho conjuntos de expresión génica (colon, leucemia, linfoma, SRBCT, sistema nervioso central, ovario y dos más). Sobre Fashion-MNIST el filtrado baja de 724 a 372 variables y la exactitud sube de 0,935 a 0,982.

Implementación con software libre

Para una implementación con software libre se eligió DEAP (*Distributed Evolutionary Algorithms in Python*), presentado por Fortin et al. (2012): es la biblioteca de la clase 7 y deja explícitos los componentes del pseudocódigo. El módulo `creator` define el tipo de fitness (con `weights=(-1.0,)` para minimizar) y el de individuo; el `toolbox` registra los operadores: `selTournament` para la selección, `cxOnePoint` o `cxBlend` para el cruce, `mutFlipBit` o `mutGaussian` para la mutación y la función de evaluación del problema; `algorithms.eaSimple` ejecuta el bucle con `cxpb`, `mutpb` y

`ngen`. El caso binario y el real comparten el esqueleto y solo cambian los atributos del individuo y los operadores; así está armada la comparación anterior (Apéndice A.3).

En R, el paquete GA (Scrucca, 2013) resuelve el mismo problema con una sola llamada: `ga(type = "real-valued", fitness = function(x) -f(x), lower = c(-5.12, -5.12), upper = c(5.12, 5.12), popSize = 50, pcrossover = 0.8, pmutation = 0.1, maxiter = 100)`; con `type = "binary"` y el argumento `nBits` pasa a la codificación binaria. El paquete maximiza, por eso la aptitud es $-f$. Se prefirió DEAP para la corrida porque cada operador queda a la vista, mientras que GA los trae ya elegidos por defecto.

Fuentes del Ejercicio 4. Material de cátedra (Volpacchio, s.f.): presentación `Algoritmos geneticos Universidad Austral.pdf` (ciclo básico, ruleta, representación de 22 bits y teorema del esquema) y notebook de la clase 7, mitad evolutiva (DEAP y máscara binaria para variables). Holland (1975) para el esquema original y Goldberg (1989) para la codificación binaria y el efecto de los operadores; Michalewicz (1996) para los AG reales y la estructura del pseudocódigo; Davis (1985), Montana y Davis (1989) y Feng (2024) para los tres casos de aplicación; Fortin et al. (2012) para DEAP y Scrucca (2013) para el paquete GA de R. Código propio: Apéndice A.3.

Ejercicio 5 · Componentes principales y kernels

Explique brevemente la utilización de Componente principal en Machine Learning. Investigue la implicancia de utilizar Kernel en Componentes principales (citar fuentes y explique con sus palabras).

¿Qué significa el "Truco del Kernel"? De ejemplos de los kernel más utilizados.

¿Investigue si existen métodos sencillos que se pueden utilizar para seleccionar el Kernel apropiado en un determinado data set? Mencione fuentes y conclusiones brevemente.

PCA en Machine Learning

El análisis de componentes principales (PCA) es una técnica lineal de reducción de dimensionalidad. Busca combinaciones lineales de las variables originales, ortogonales entre sí, que concentran la mayor varianza posible: esas direcciones son los autovectores de la matriz de covarianzas, y la varianza que captura cada una es su autovalor. La primera componente captura la mayor varianza, la segunda la mayor varianza restante, y así sucesivamente; al retener las primeras k se obtiene una representación de menor dimensión que conserva un porcentaje conocido de la varianza total.

En Machine Learning el PCA reduce la cantidad de variables de entrada p, lo que baja el costo de entrenamiento y acota la capacidad del modelo (la riqueza de la familia de funciones que puede ajustar): para un clasificador lineal la dimensión VC es $d = p + 1$, el $d = k + 1$ del Ejercicio 3, y bajar p relaja la regla $d/l < 0{,}05$. Decorrelaciona las variables y filtra ruido, porque las últimas componentes suelen capturar variación espuria y colinealidad. Y sirve para visualizar: la proyección sobre las dos o tres primeras componentes permite inspeccionar si hay grupos antes de un clustering, uso que aparece en `Preparation for Clustering.pdf` (Volpacchio, s.f.). El PCA no selecciona variables, construye otras nuevas como combinación de todas; por eso una componente es más difícil de interpretar que una variable original. Su limitación estructural es que solo captura relaciones lineales.

Kernel PCA figura en el material sin desarrollo: el apunte de taxonomía de la clase 4 lo lista como variante no lineal del PCA, y `t_sne.pdf` de la clase 7 lo menciona entre las alternativas no lineales. De la selección de kernel solo hay prueba y error sobre datos de validación, con la herramienta de SVM del Ejercicio 2. No corresponde tomar el PDF `Transformación de clústeres no lineales` como Kernel PCA: ese material mapea clusters con una ϕ explícita y no diagonaliza una matriz de kernel.

El truco del kernel

Un kernel es una función $K(x, x')$ que devuelve el producto interno de dos observaciones en un espacio de características: un espacio de dimensión mayor que el original, alcanzado por una transformación ϕ, en el cual un problema no lineal puede volverse lineal. El ejemplo de la clase de SVM es el problema XOR (dos clases ubicadas en esquinas opuestas de un cuadrado), que se vuelve linealmente separable al agregar la característica $x_1 x_2$. Calcular ϕ puede ser muy costoso, y el espacio puede ser de dimensión infinita. El truco consiste en notar que muchos algoritmos, SVM y PCA entre ellos, solo usan los datos a través de productos internos; basta entonces con reemplazar $\langle \phi(x), \phi(x') \rangle$ por $K(x, x')$, evaluada en el espacio original, sin construir nunca la transformación. Para que exista una ϕ compatible, K debe cumplir la condición de Mercer definida en el Ejercicio 2.

Los kernels más utilizados, con la notación del Ejercicio 2 y de la clase de SVM (Volpacchio, s.f.), son:

- lineal, $K(x, x') = \langle x, x' \rangle$: equivale a no transformar y se usa cuando la relación es aproximadamente lineal o hay muchas variables;
- polinómico de grado q, $K(x, x') = \langle x, x' \rangle^q$: incorpora todas las interacciones de hasta q variables;
- de base radial (RBF o gaussiano), $K(x, x') = \exp(-\|x - x'\|^2/(2\sigma^2))$: la similitud decae con la distancia, y σ fija qué tan rápido;
- sigmoide, $K(x, x') = \tanh(\kappa \langle x, x' \rangle - \theta)$, que no cumple la condición de Mercer para toda combinación de κ y θ; por eso no se lo toma como opción por defecto.

La suma, el producto y el escalado por una constante positiva de kernels válidos dan kernels válidos, lo que permite construir kernels a medida de un problema.

Kernel PCA

Kernel PCA es la aplicación del truco del kernel al PCA, propuesta por Schölkopf et al. (1998). En lugar de diagonalizar la matriz de covarianzas, de dimensión $p \times p$, se diagonaliza la matriz de kernel de dimensión $n \times n$, cuyos elementos son $K(x_i, x_j)$, previamente centrada: a cada elemento se le restan las medias de su fila y de su columna y se le suma la media general, lo que equivale a centrar los datos en el espacio de características sin construirlo. Las proyecciones sobre las componentes también se calculan con evaluaciones del kernel. La implicancia central es que las componentes dejan de ser combinaciones lineales de las variables originales y pasan a capturar estructura no lineal: datos dispuestos sobre curvas o anillos concéntricos, que el PCA lineal no separa, pueden quedar ordenados a lo largo de las primeras componentes no lineales.

De ahí salen consecuencias prácticas. El costo pasa a depender del número de observaciones n y no de p, lo que encarece el método en conjuntos grandes. Se pueden extraer hasta n componentes, más que las p del caso lineal. Y se pierde la reconstrucción directa: un punto del espacio de características no siempre tiene pre-imagen exacta en el espacio original, y las componentes ya no tienen cargas sobre las variables que se puedan leer. Kernel PCA extiende al PCA; no lo reemplaza, y con el kernel lineal se recupera el PCA clásico.

Cómo elegir el kernel

Sobre la selección del kernel, la bibliografía no ofrece un método universal, pero sí procedimientos sencillos que se complementan. La guía de Hsu et al. (2003) decide por las dimensiones del conjunto: recomienda el kernel lineal cuando la cantidad de variables es grande frente a la de observaciones, porque llevar los datos a un espacio mayor no agrega separabilidad, y también cuando las dos cantidades son grandes, por costo; si hay muchas más observaciones que variables, el candidato es el RBF.

La validación cruzada sobre una grilla de kernels y de sus hiperparámetros decide al final: para cada combinación se estima el error en los bloques de validación y se queda la mejor. Hsu et al. (2003) recomiendan empezar por el RBF, que con parámetros bien elegidos cubre al lineal como caso límite, y buscar C y $\gamma = 1/(2\sigma^2)$ en una grilla exponencial, como se hizo en el Ejercicio 10.

El alineamiento kernel-objetivo (*kernel-target alignment*) de Cristianini et al. (2002) evita entrenar el modelo para cada candidato: mide el parecido entre la matriz de kernel K y la matriz ideal $yy^\top$ construida desde las etiquetas $y_i \in \{-1, +1\}$, $A = \langle K, yy^\top \rangle_F / (\|K\|_F \, \|yy^\top\|_F)$, donde $\langle \cdot, \cdot \rangle_F$ suma los productos elemento a elemento. Un A alto indica que el kernel ve parecidas a las observaciones de la misma clase y distintas a las de clases distintas; se elige el kernel de mayor alineamiento. Una variante más elaborada es el aprendizaje de múltiples kernels (*multiple kernel learning*), que combina varios kernels con pesos que se aprenden en el entrenamiento (Gönen & Alpaydın, 2011).

El atajo se puso a prueba sobre el conjunto de clasificación del Ejercicio 10, con la misma partición y el mismo escalado: 455 casos de Breast Cancer, con el maligno como clase positiva. El kernel lineal da $A = 0{,}5255$, muy por encima del resto; el mejor RBF es el de $\gamma = 0{,}1$ con 0,3692, y los polinómicos de grado 2 y 3 quedan en 0,0419 y 0,0357. La validación cruzada del Ejercicio 10 eligió $\gamma = 0{,}01$, cuyo alineamiento es 0,2172: **el atajo no reproduce la elección del hiperparámetro**. Donde sí coincide es en la familia, y ese acuerdo vale, porque el alineamiento más alto es el del kernel lineal y respalda con una medición la lectura del Ejercicio 10 de que las clases son casi linealmente separables en el espacio estandarizado. Se interpreta que el alineamiento sirve para descartar familias antes de entrenar, no para ajustar un parámetro dentro de una.

En la práctica conviene empezar por el RBF y recurrir al alineamiento cuando entrenar cada candidato sale caro. Para Kernel PCA la elección es más difícil, porque no hay etiquetas contra las cuales validar: en la práctica el kernel se ajusta por el error de reconstrucción o por el desempeño de la tarea posterior que usa las componentes.

Fuentes del Ejercicio 5. Material de cátedra (Volpacchio, s.f.): `Preparation for Clustering.pdf` (PCA como visualización previa al clustering) y presentación `SVM Universidad Austral.pdf` (espacio de características, truco del kernel y kernels lineal, polinómico, RBF y sigmoide). Kernel PCA y la selección sistemática de kernel no tienen material de clase: Schölkopf et al. (1998) para Kernel PCA; Hsu et al. (2003) para la regla de dimensiones y la grilla exponencial; Cristianini et al. (2002) para el alineamiento kernel-objetivo; Gönen y Alpaydın (2011) para el aprendizaje de múltiples kernels. Código propio: Apéndice A.4.

Ejercicio 6 · Análisis de supervivencia

Análisis de supervivencia. Explique porque este enfoque es superior a los métodos de regresión para estimar días a un evento. Qué tipos existen y qué implementaciones en Python/R conoce.

Por qué supera a la regresión

En los problemas de tiempo hasta un evento, la variable de interés es el tiempo T hasta que ocurre algo definido: la falla de un equipo, la baja de un cliente, la reincidencia de un liberado, la recaída de un paciente. En la notación de la clase 7, el dato de cada individuo no es un número sino la pareja (T_i, δ_i): T_i es el tiempo observado y δ_i el indicador de evento, con $\delta_i = 1$ si el evento ocurrió y $\delta_i = 0$ si la observación está censurada (Volpacchio, s.f.; Kleinbaum & Klein, 2012). La censura por derecha, la más frecuente, se produce cuando el seguimiento termina antes del evento. Un cliente seguido 100 días que sigue activo aporta $T_i = 100$ y $\delta_i = 0$: solo se sabe que su baja, si ocurre, será posterior al día 100. También existen la censura por izquierda y por intervalo; la clase trabaja con la de derecha.

Una regresión sobre los días hasta el evento no tiene forma correcta de tratar esos casos. Si se los descarta, se pierde información y la muestra queda sesgada hacia los eventos tempranos, porque los individuos de mayor duración son justamente los que más se censuran. Si se los conserva con T_i como si fuera el tiempo del evento, los tiempos se subestiman. El análisis de supervivencia resuelve el problema en la verosimilitud, la función que mide qué tan compatibles son los datos con los parámetros del modelo: un caso con evento aporta la densidad en T_i y un caso censurado aporta la probabilidad $P(T > T_i)$, y la información parcial se aprovecha en lugar de descartarse o falsearse.

A esa ventaja se suma el objeto que se estima. La cantidad central es la función de supervivencia $S(t) = P(T > t)$, la probabilidad de que el evento no haya ocurrido hasta el tiempo t, junto con la tasa de riesgo instantánea o *hazard* $h(t \mid X)$, la probabilidad por unidad de tiempo de que el evento ocurra en t entre quienes llegaron a t sin él. Una regresión devuelve un único valor esperado; la curva $S(t)$ responde preguntas a cualquier horizonte, por ejemplo qué fracción de clientes sigue activa a los 90 días.

Tipos

Los tipos de análisis se ordenan por la cantidad de supuestos. El estimador no paramétrico de Kaplan-Meier (Kaplan & Meier, 1958) construye la supervivencia como producto de supervivencias condicionales en cada tiempo de evento, $\hat{S}(t) = \prod_{t_i \le t}(1 - d_i/n_i)$, donde d_i cuenta los eventos en t_i y n_i es el conjunto en riesgo justo antes de t_i. No admite covariables continuas: los grupos se comparan por estratificación, y el test de log-rank decide si dos curvas difieren más de lo esperable por azar.

El enfoque semiparamétrico es el modelo de riesgos proporcionales de Cox (1972), $h(t \mid X) = h_0(t)\, e^{\beta^\top X}$, donde el riesgo basal $h_0(t)$ no se estima paramétricamente y los coeficientes β salen de la verosimilitud parcial: en cada tiempo de evento se toma la probabilidad de que sea justamente ese individuo, y no otro del conjunto en riesgo, el que tenga el evento, un cociente en el que $h_0(t)$ se cancela. El exponencial de cada coeficiente es un *hazard ratio*: cuánto multiplica esa covariable el riesgo instantáneo. El supuesto de riesgos proporcionales se verifica con la prueba de Grambsch y Therneau (1994) sobre los residuos de Schoenfeld (la diferencia entre la covariable de quien tuvo el evento y su promedio en el conjunto en riesgo); el desempeño se mide con el índice de concordancia (C-index), la probabilidad de que el modelo ordene bien los tiempos de un par de individuos.

El enfoque paramétrico son los modelos de tiempo de falla acelerado (AFT), que modelan $\log T = \beta^\top X + \sigma\varepsilon$, con σ un parámetro de escala y ε un error cuya distribución define el modelo (valor extremo si T es Weibull, logística si T es log-logístico). La lectura se invierte respecto de Cox: un coeficiente positivo alarga el tiempo hasta el evento. Se los prefiere cuando el supuesto de riesgos proporcionales no se cumple o cuando se necesitan tiempos absolutos; entre modelos paramétricos se comparan con el criterio de Akaike (AIC), que penaliza la verosimilitud por la cantidad de parámetros. Por último, hay extensiones de aprendizaje automático, como los *Random Survival Forests* (Ishwaran et al., 2008), que no están en la clase 7 y se evalúan también por C-index.

La ventaja se midió sobre los datos Rossi de la clase 7 (Rossi et al., 1980): 432 liberados seguidos 52 semanas, de los cuales 114 fueron re-arrestados y 318 (74 %) quedaron censurados en la semana 52, el fin del estudio. Los dos atajos de regresión dan respuestas incompatibles: promediar solo a los re-arrestados da 28,7 semanas, y tratar la semana 52 como si fuera el re-arresto da 45,9. El primero mira solo al 26 % que tuvo el evento; el segundo no es un tiempo hasta el evento sino una media truncada en 52 semanas, que coincide con el área bajo la curva de Kaplan-Meier hasta ese horizonte (45,9) porque toda la censura cae en la misma semana. Kaplan-Meier da la respuesta que la regresión no puede expresar: $\widehat{S}(52) = 0{,}736$, la mediana no se alcanza dentro del seguimiento, y lo correcto es informar que más de la mitad no reincide en 52 semanas, no inventar una cantidad de días.

El efecto de una covariable muestra lo mismo. Una regresión lineal de las semanas sobre las siete covariables, ajustada solo con los re-arrestados, asigna a la ayuda financiera (`fin`) un coeficiente de -0,41 semanas, un efecto casi nulo y, en todo caso, hacia un re-arresto más temprano. Cox, con los 432 individuos, estima un *hazard ratio* de 0,684 (IC 95 %: 0,470 a 0,996; $p = 0{,}047$): la ayuda reduce el riesgo instantáneo en un 32 %. Se interpreta que la regresión no lo ve porque la ayuda actúa sobre quiénes llegan a la semana 52 sin re-arresto, que son los casos que descarta. El C-index del modelo de Cox es 0,64.

Tabla 4: Tipos de análisis de supervivencia e implementaciones.

Tipo	Supuesto	Python	R
No paramétrico	Ninguno sobre la distribución de T	lifelines: KaplanMeierFitter, logrank_test	survival: survfit, survdiff; survminer para gráficos
Semiparamétrico	Riesgos proporcionales	lifelines: CoxPHFitter; statsmodels: PHReg	survival: coxph
Paramétrico (AFT)	Distribución de T (Weibull, log-logística)	lifelines: WeibullAFTFitter, LogLogisticAFTFitter	survival: survreg; flexsurv
Aprendizaje automático	Ninguno; ensamble de árboles	scikit-survival: RandomSurvivalForest	randomForestSRC

Nota. Elaboración propia; funciones de lifelines tomadas del notebook de la clase 7.

Implementaciones en Python y R

En Python, lifelines (Davidson-Pilon, 2019) es la biblioteca de la clase y la de la corrida anterior; scikit-survival (Pölsterl, 2020) adapta modelos de aprendizaje automático al dato censurado con la interfaz de scikit-learn. En R, el objeto `Surv(tiempo, evento)` del paquete survival (Therneau & Grambsch, 2000) alimenta las funciones de la Tabla 4.

Fuentes del Ejercicio 6. Material de cátedra (Volpacchio, s.f.): notebook de la clase 7, parte de supervivencia (Kaplan-Meier, log-rank, Cox y AFT sobre los datos Rossi). Kleinbaum y Klein

(2012) para la censura y los tipos de análisis; Kaplan y Meier (1958) para el estimador no paramétrico y Cox (1972) para el modelo de riesgos proporcionales; Grambsch y Therneau (1994) para el diagnóstico de ese supuesto; Ishwaran et al. (2008) para los bosques de supervivencia; Rossi et al. (1980) para los datos; Davidson-Pilon (2019), Pölsterl (2020) y Therneau y Grambsch (2000) para las implementaciones. Código propio: Apéndice A.5.

Ejercicio 7 · Clustering y t-SNE

Mencione las técnicas de pre-proceso que existen en clustering para visualizar la potencial existencia de grupos. Explique brevemente 1 de ellas. Habiendo aplicado alguna técnica de clustering y obteniendo los grupos finales, ¿de qué manera puede estimar qué diferencias existen en los perfiles de los grupos obtenidos? Explicar los resultados. Ayuda: utilice el último capítulo de la guía de clustering (Assessment Clustering). ¿En qué consiste el algoritmo t-SNE? ¿Para qué se utiliza? De un ejemplo de implementación en Python. ¿Qué limitantes tiene? ¿Qué ventajas tiene?

Técnicas de pre-proceso

Todo algoritmo de clustering devuelve una partición, haya grupos en los datos o no. Por eso conviene mirar antes si esa estructura existe, y como los datos suelen tener más dimensiones de las que entran en un gráfico, mirar quiere decir proyectarlos a dos dimensiones.

La guía de clustering de la cátedra trae tres ayudas gráficas. Los gráficos de componentes principales (PRINCOMP) proyectan sobre las dos primeras componentes. El escalado multidimensional (MDS) ubica los puntos en dos dimensiones con la mayor conservación posible de las distancias originales. El discriminante canónico (CANDISC) busca las direcciones que maximizan la separación entre grupos, pero las necesita ya definidas: sirve para mirar una partición que ya se tiene, no para descubrirla, y por eso reaparece más abajo como tercer paso del perfilado. A esas tres hoy se les suman t-SNE y UMAP (McInnes et al., 2018), que para este fin desplazaron a MDS.

El capítulo trata además tres pasos que no son de visualización pero deciden qué muestran los gráficos. VARCLUS selecciona variables y elimina colinealidad. STDIZE estandariza, con énfasis en la escala y en el efecto del orden de los datos sobre la inicialización. Y ACECLUS estima la matriz de covarianza dentro de los grupos sin conocerlos, y transforma los datos para que la distancia euclídea en el espacio transformado se comporte como una distancia de Mahalanobis en el original, la euclídea después de descontar la covarianza, útil con grupos elipsoidales y rotados respecto de los ejes.

La técnica que se desarrolla es la primera. En los gráficos de componentes principales se estandarizan las variables, se calculan los autovectores de la matriz de correlación y se proyectan las observaciones sobre las dos primeras componentes. El fundamento para este uso es que, si existen grupos separados y la separación tiene una componente lineal apreciable, las direcciones de máxima varianza tienden a alinearse con ella y los grupos aparecen como nubes distinguibles. La ventaja es el costo bajo y el hecho de que las cargas indican en función de qué variables se separan los grupos. La limitación es la misma que se señala en el Ejercicio 5: al tratarse de una proyección lineal, no detecta grupos definidos por estructuras curvas, y un conjunto de anillos concéntricos aparece como una nube sin estructura. Ese es precisamente el problema que motiva a t-SNE.

Perfilado de los grupos obtenidos

El perfilado es la generación de descripciones de cada grupo a partir de las variables de entrada, y la guía lo ordena en tres pasos: los vectores de medias de cada grupo, el MANOVA y, recién después, el ANOVA por variable. El MANOVA es un análisis de varianza con varias variables respuesta a la vez: pregunta si los grupos difieren en el vector completo y no en una variable. Va primero porque así controla el error de tipo I a nivel del experimento, en lugar de inflarlo con una prueba por variable. El ANOVA va acompañado del análisis discriminante canónico, que identifica qué variables producen las diferencias; a la inversa es igual de útil, porque las que no difieren entre grupos pueden podarse del conjunto de entrada.

La propia guía advierte que los resultados de MANOVA y ANOVA deben interpretarse con cautela, porque el clustering viola supuestos clave de esas pruebas: los grupos no se definieron de forma independiente de los datos sino para maximizar la separación, así que los valores p quedan sesgados hacia la significación. Valen como medida del tamaño de las diferencias, no como prueba de hipótesis. La guía deja ahí el problema, así que a los tres pasos se agrega un clasificador contra la etiqueta de grupo, tratada como clase dada, y se lee su importancia de variables: no hay supuesto distribucional que violar, la salida ya es la descripción del grupo y la exactitud mide cuán separables son. Se descarta la alternativa de pruebas no paramétricas variable por variable, porque el sesgo viene de que los grupos se construyeron con estos mismos datos y no del supuesto de normalidad.

Se aplicó a los diez grupos de k-means sobre los dígitos que se describen más abajo; con 64 píxeles por imagen una tabla de 10 por 64 sería ilegible, y el perfilado va sobre las cinco primeras componentes principales.

Tabla 5: Perfiles de los diez grupos de k-means sobre las cinco primeras componentes.

Grupo	n	PC1	PC2	PC3	PC4	PC5
0	275	0,50	0,77	2,33	-0,05	-0,82
1	432	1,80	-0,03	-2,17	1,11	-0,24
2	29	-5,37	5,98	-1,30	0,03	7,07
3	160	-4,67	-0,72	1,17	-0,89	-1,03
4	178	-1,63	-1,96	-3,05	-1,69	-0,59
5	178	-0,55	3,52	0,51	-0,34	-0,24
6	77	2,62	-1,87	2,24	2,09	3,17
7	182	-1,26	-3,85	1,13	0,06	1,17
8	110	-2,45	1,80	0,01	3,00	-0,49
9	176	3,82	0,64	1,24	-2,62	0,21

Nota. Elaboración propia a partir de `codigo/resultados/e7/perfiles_kmeans.csv`.

Lo que más dice de la tabla son los tamaños. Con diez dígitos que aparecen unas 180 veces cada uno, un grupo de 432 casos fusiona al menos dos clases y otro de 29 es un fragmento, no una clase. Antes de mirar una sola media ya se sabe que la partición no reproduce las diez clases, lo cual es coherente con el índice de Rand ajustado de 0,476 que se reporta más abajo. Después están los apartamientos: un grupo se aleja hasta 5,98 en la segunda componente y 7,07 en la quinta, que es el perfil de un grupo de atípicos, mientras que otros dos se oponen sobre la primera componente con -4,67 contra 3,82.

El MANOVA, que va primero, se calculó con las matrices de dispersión dentro de los grupos y total: el Λ de Wilks es 0,0034, y la aproximación de Bartlett lo lleva a $\chi^2(45) = 10.180$. Con 1 como ninguna separación y 0 como separación total, los diez grupos difieren en el vector y no en una sola componente.

El ANOVA por componente, con los diez grupos como factor, da valores F de 730,4, 515,8, 442,8, 217,0 y 247,3 para las cinco componentes. Más que las magnitudes importa que el orden de los F no coincide con el de las componentes: la quinta separa mejor los grupos que la cuarta pese a explicar menos varianza total. Esa es exactamente la diferencia entre varianza total y varianza entre grupos, y el discriminante canónico la pone en números: sus dos primeros ejes concentran el 61,7 % de la separación entre grupos, contra el 21,6 % de la varianza total que explican las dos

primeras componentes principales. Los valores p que acompañan a esos F no se reportan, por la advertencia del párrafo anterior.

Queda el agregado al esquema de la guía. Un bosque aleatorio de 500 árboles contra la etiqueta de grupo, sobre esas mismas cinco componentes, acierta el 91,5 % fuera de la bolsa, y su importancia las ordena PC1, PC3, PC2, PC4, PC5. El orden discrepa con los F en dos pares, y uno de ellos es justo donde el párrafo anterior puso el acento, porque el F mide la separación de una componente por vez y la importancia mide lo que cada una agrega dado lo que aportan las otras: la quinta separa bien sola, pero casi todo eso ya está en las tres primeras. El orden se repitió idéntico en las 8 semillas, con el OOB entre 0,9126 y 0,9204. Ninguna de las cinco tiene nombre de dominio, así que el perfil dice en qué dirección difieren los grupos y no qué significan.

El algoritmo t-SNE

El apunte de la clase 8 abre con una aclaración que evita la mayor parte de los malentendidos: t-SNE (*t-distributed stochastic neighbor embedding*) no es un algoritmo de clustering. Transforma el espacio para que las estructuras latentes se vuelvan visibles; el agrupamiento viene después, y sobre el espacio original.

En alta dimensión las distancias euclídeas al cuadrado se convierten en probabilidades mediante un kernel gaussiano centrado en cada punto x_i, $p_{j|i} \propto \exp(-\|x_i - x_j\|^2/2\sigma_i^2)$, simetrizadas como $p_{ij} = (p_{j|i} + p_{i|j})/2n$. En baja dimensión, y ahí está lo que da el nombre al método, se usa una distribución t de Student con un grado de libertad sobre las posiciones y_i del mapa, $q_{ij} \propto (1 + \|y_i - y_j\|^2)^{-1}$, cuyas colas pesadas resuelven el problema de aglomeración, la tendencia de los puntos moderadamente distantes a amontonarse al proyectar. La función de costo es la divergencia de Kullback-Leibler, $KL(P\|Q) = \sum_{i \neq j} p_{ij} \log(p_{ij}/q_{ij})$, cuya asimetría es constitutiva: penaliza con fuerza separar en el mapa a vecinos que estaban cerca y tolera que puntos lejanos queden próximos, de donde se sigue que preserve la estructura local y no la global. La optimización es un descenso de gradiente sobre las coordenadas del mapa (van der Maaten & Hinton, 2008), que el apunte de la clase 8 inicializa con los componentes principales de un PCA previo para acelerar la convergencia.

El único hiperparámetro relevante es la perplejidad, que funciona como ancho de banda del kernel. Se define como $2^{H(P_i)}$, con H la entropía de Shannon de la vecindad del punto, de manera que 4,9 bits equivalen a unos 30 vecinos efectivos. El σ_i de cada punto sale por búsqueda binaria hasta alcanzar esa perplejidad, lo cual lo vuelve adaptativo: chico en zonas densas y ancho en zonas dispersas, con valores usuales de 5 a 50. La variante de Barnes-Hut agrupa los puntos lejanos y los trata como una masa única, con lo cual la complejidad baja de $O(N^2)$ a $O(N \log N)$ y el método se vuelve utilizable más allá de unos pocos miles de puntos (van der Maaten, 2014).

Se utiliza para la visualización exploratoria de datos de alta dimensión: inspección de embeddings, exploración previa a un clustering, control visual de una partición y detección de subpoblaciones. El apunte propone un pipeline de tres pasos que consiste en explorar con t-SNE o UMAP sobre datos estandarizados, modelar sobre el espacio original con un algoritmo que no exija grupos esféricos, como DBSCAN (Ester et al., 1996) o su variante jerárquica HDBSCAN (Campello et al., 2013), y validar mediante la superposición de las etiquetas obtenidas sobre la proyección.

El ejemplo (apéndice A.6) aplica ese pipeline sobre los dígitos manuscritos de `scikit-learn`, 1797 imágenes de 8 por 8 píxeles. Se estandariza con `StandardScaler`, se reduce con `PCA(n_components=30)`, se proyecta con `TSNE(perplexity=30, init="pca")` y se agrupa

con `DBSCAN` o `KMeans` sobre las 30 componentes, nunca sobre el embedding, que es lo que el apunte admite al hablar de agrupar sobre el espacio original "o sobre una reducción PCA previa si la dimensionalidad es extrema". El PCA conserva el 89,3 % de la varianza, y las distancias son las de los datos.

Resultados sobre los dígitos

El mapa, con las etiquetas superpuestas sobre la proyección, está en el apéndice A.6. Para medir su calidad se usa el coeficiente de silueta, que compara para cada punto su distancia media al propio grupo contra la del grupo vecino más cercano: va de -1 a 1, y cuanto más alto, más compactos y separados están los grupos. Sobre las clases verdaderas dio 0,396 con perplejidad 5, 0,486 con 30 y 0,479 con 50, es decir que mejora al pasar de 5 a 30 y después se estanca.

DBSCAN agrupa por densidad: un punto es núcleo si tiene al menos 8 puntos, él incluido, a distancia menor que un radio ε; los núcleos vecinos se encadenan y el resto queda como ruido. El acuerdo con los dígitos se mide con el índice de Rand ajustado (ARI), la proporción de pares que ambas particiones dejan juntos o separados, corregida por el azar: 1 si coinciden, 0 si es el del azar. El radio se ubicó con la distancia al octavo vecino (gráfico de distancia k; Ester et al., 1996), de mediana 3,77 y cuantiles del 10 y del 90 % de 2,81 y 5,18; una primera grilla empezaba por encima de la mediana y no hallaba grupos. Entre los 16 valores barridos, de 2,00 a 20,00, el mejor ARI sale con $\varepsilon = 3{,}50$: 10 grupos, 713 puntos de ruido (39,7 %) y ARI de 0,247 (0,623 sin el ruido).

Con $\varepsilon = 2{,}00$ quedan 1765 de los 1797 puntos como ruido, y desde $\varepsilon = 5{,}00$ todo se funde en un grupo (ARI de 0,001): bajar el radio parte grupos pero manda puntos al ruido, y subirlo los recupera pero funde dígitos. La causa probable es un radio global frente a una densidad que varía entre dígitos, más que la concentración de distancias del apunte.

K-means con $k = 10$ sobre las componentes dio un ARI de 0,476, y sobre las dos coordenadas del embedding dio 0,845, muy por encima. La tentación de quedarse con ese número es la trampa: no mide la capacidad de k-means para descubrir estructura sino el trabajo que ya hizo t-SNE al separarla: sirve como validación visual, no como modelo.

Ventajas y limitantes

Entre las ventajas, t-SNE captura estructura no lineal a diferencia de PCA y desenreda superficies curvas (variedades) que MDS superpone; preserva la vecindad local, que es lo que separa visualmente los grupos; las colas pesadas de la t resuelven la aglomeración; y tiene un solo hiperparámetro relevante.

Entre las limitantes, la principal es que no es clustering: los grupos visuales hay que confirmarlos sobre el espacio original. Las distancias entre grupos, los ejes y los tamaños relativos no significan nada, por la asimetría de la divergencia. La función de costo no es convexa, así que cada corrida da un mapa distinto y hay que fijar la semilla. Es sensible a la perplejidad, y que la silueta de arriba apenas se mueva entre 30 y 50 no lo desmiente, porque esa mide clases ya conocidas. Tampoco es un modelo entrenado: la implementación estándar no proyecta observaciones nuevas sin recalcular todo, y por eso UMAP lo reemplaza ahí. Y con perplejidad muy baja puede producir grupos nítidos a partir de datos sin estructura.

Fuentes del Ejercicio 7. Material de cátedra (Volpacchio, s.f.): *Preparation for Clustering*, *Assessing Clustering Results* y `TSNE_Perplejidad_Clustering_NoLineal` (clase 8). Van der Maaten y Hinton (2008) y van der Maaten (2014) para t-SNE; Ester et al. (1996) y Campello et al. (2013) para DBSCAN; McInnes et al. (2018) para UMAP. Código: A.6.

Ejercicio 8 · AdaBoost y Gradient Boosting

Que diferencia existen entre el algoritmo Adaboost y Gradiente Boosting. Explique brevemente. De una explicación intuitiva de cómo trabaja el Gradiente Boosting.

Diferencias entre AdaBoost y Gradient Boosting

Los dos son métodos de boosting y comparten el esquema que describe la presentación de la clase 6: combinar las salidas de muchos clasificadores débiles para producir un comité que funciona como clasificador fuerte. Un clasificador débil es una regla de decisión sencilla e inestable, pero que acierta mejor que el azar, es decir con $P(\text{error}) < 0{,}5$. El método es secuencial, porque cada modelo se entrena después del anterior y mira sus errores para decidir en qué concentrarse, a diferencia de bagging, donde los modelos se entrenan en paralelo y sin mirarse.

La presentación ubica a los dos dentro de los métodos de perturbar y combinar y, más específicamente, entre los algoritmos arcing (*adaptive resampling and combining*), en contraposición a bagging, que perturba mediante bootstrap. La etiqueta encaja con AdaBoost, que repondera de manera adaptativa las observaciones en cada ronda, pero le queda grande a Gradient Boosting, que no repondera ni remuestrea nada: lo que cambia de ronda en ronda es el objetivo que se ajusta. La presentación los agrupa igual, y la diferencia aparece dos párrafos más abajo.

AdaBoost cambia la importancia de las observaciones y mantiene fijo el objetivo; Gradient Boosting cambia el objetivo y mantiene fija la importancia de las observaciones. Son dos maneras distintas de indicarle al siguiente modelo dónde están los errores del anterior. De ahí se desprende el resto: AdaBoost nació para clasificación y tiene una función de pérdida implícita, la exponencial $e^{-yF(x)}$, mientras que Gradient Boosting admite cualquier pérdida diferenciable y sirve igual para regresión; AdaBoost pondera cada hipótesis con un α_t que sale en forma cerrada y Gradient Boosting obtiene su coeficiente por minimización; y AdaBoost es más sensible a los valores atípicos, porque un atípico mal clasificado gana peso ronda tras ronda, mientras que en Gradient Boosting esa sensibilidad se regula con la elección de la pérdida.

El pseudo-residuo es el objetivo nuevo de cada ronda: el gradiente de la pérdida respecto de la predicción, con el signo cambiado y evaluado en el modelo acumulado hasta esa ronda, $\tilde{y}_{im} = -[\partial L(y_i, F(x_i))/\partial F(x_i)]$. Con pérdida cuadrática se reduce al residuo común $y_i - F_{m-1}(x_i)$, y de ahí viene el nombre: es el residuo cuando la pérdida es la cuadrática y un sustituto del residuo cuando no lo es. El *shrinkage* es el factor ν que multiplica la contribución de cada modelo débil antes de sumarla al acumulado, para que cada corrección sea parcial.

En AdaBoost (Freund & Schapire, 1997) la primera ronda reparte peso uniforme sobre las N observaciones. En cada ronda se entrena el clasificador débil sobre esa distribución de pesos, se mide su error ponderado ε_t y se le asigna la ponderación $\alpha_t = \frac{1}{2}\log((1-\varepsilon_t)/\varepsilon_t)$, con la que entra al voto final $H(x) = \text{signo}(\sum_t \alpha_t h_t(x))$. Después el peso de cada observación se multiplica por $e^{-\alpha_t}$ si quedó bien clasificada y por $e^{+\alpha_t}$ si quedó mal, y se renormaliza, para que el algoritmo base de la ronda siguiente se focalice sobre las que se erraron. La fórmula de α_t se lee sola: si el error es chico la hipótesis pesa mucho en el voto, y si se acerca a 0,5 el peso tiende a cero y la hipótesis prácticamente no cuenta.

La presentación reproduce un ejemplo con semiplanos verticales u horizontales como clasificador base. Las tres rondas se rehicieron con la fórmula.

Tabla 6: Las tres rondas de AdaBoost de la presentación, verificadas.

Ronda	Error	Ponderación publicada	Ponderación recalculada	Observación
1	0,30	0,42	0,4236	coincide al redondear
2	0,21	0,65	0,6625	redondea a 0,66, no a 0,65
3	0,14	0,92	0,9076	redondea a 0,91, no a 0,92

Nota. Elaboración propia. El error es el ε_t ponderado de la ronda y la ponderación, el $\alpha_t = \frac{1}{2}\log((1-\varepsilon_t)/\varepsilon_t)$ con el que la hipótesis entra al voto. Los seis valores publicados se leen de las diapositivas del ejemplo.

Dos de las tres rondas no cierran: el error que produce exactamente 0,65 es 0,2142 y el que produce 0,92 es 0,137, de manera que lo más probable es que las cuentas originales se hayan hecho con los errores sin redondear. La diferencia es de un centésimo y no cambia nada, pero queda constancia de que se verificó. El clasificador base, un único corte paralelo a un eje, es un *stump*, un árbol de un solo nivel, y alcanza.

Gradient Boosting se formula como $F_m(x) = F_{m-1}(x) + \beta_m h(x; a_m)$ y se resuelve en dos etapas: primero $h(x)$ se ajusta por mínimos cuadrados a los pseudo-residuos y después, con h ya fijo, el β óptimo se obtiene por minimización. Lo que se gana es que la función de pérdida deja de estar cableada en el algoritmo y pasa a ser un parámetro que hay que elegir, entre mínimos cuadrados, mínima desviación absoluta, la de Huber (cuadrática cerca del cero y lineal en las colas, para que un atípico no domine el ajuste) o la log-verosimilitud multinomial negativa para clasificación en varias clases. El otro parámetro es el *shrinkage*, que la presentación llama tasa de aprendizaje, y Friedman (2001) estimó que valores menores a 0,1 dan los menores errores de generalización. Visto así, AdaBoost es un caso particular y no un método aparte: es el Gradient Boosting cuya pérdida es la exponencial. Esa unificación la presentación no la hace explícita, pero es la que convierte la lista de diferencias en una jerarquía.

Cómo trabaja Gradient Boosting

La explicación intuitiva de `Gradient boosting explicado` es la del golfista. Un golfista intenta embocar la pelota en el hoyo, ubicado en y. El primer golpe, $F_0(x)$, es una aproximación gruesa que en el caso más simple consiste en predecir para todos el promedio de y, y la pelota queda a cierta distancia. El golfista no vuelve al punto de partida: golpea desde donde quedó la pelota y calcula el nuevo golpe según cuánto le falta. Ese segundo golpe corrige el error del primero, el tercero corrige lo que quedó del segundo, y así sucesivamente, de manera que $F_m(x) = F_{m-1}(x) + \nu\,\beta_m h_m(x)$, donde ν contrae el paso óptimo β_m de la fórmula de arriba. Cada modelo débil no intenta predecir y sino la distancia que falta, y esa distancia es el pseudo-residuo. El ejemplo numérico son los alquileres de cinco departamentos, donde el golpe inicial es la media, 1418, y a partir de ahí cada modelo se entrena sobre el vector de residuos que dejó el anterior.

El nombre del método viene de que el gradiente de la pérdida cuadrática respecto de la predicción es $\partial L/\partial F = -(y - F)$, con lo cual el vector de residuos es el gradiente negativo y no el gradiente.

El signo importa, porque es la razón por la que sumar el residuo al modelo acumulado baja la pérdida en lugar de subirla. Ajustar los residuos es hacer descenso de gradiente, pero en el espacio de las predicciones en lugar del espacio de los parámetros, con la particularidad de que cada paso lo da un modelo débil. Eso explica la generalidad del método: si la pérdida es el error absoluto

medio, su gradiente ya no es el vector de residuos sino el vector de signos, ± 1 según si la predicción quedó por debajo o por encima, y optimizar esa pérdida implica arrancar con la mediana en lugar de la media. Cambiar la función de pérdida cambia qué se le pide ajustar al modelo débil, y nada más. Ahí está también por qué la desviación absoluta conviene cuando hay atípicos: con pérdida cuadrática el modelo débil se entrena sobre la magnitud del residuo y persigue los valores extremos, porque un residuo enorme domina el ajuste, mientras que con el vector de signos un atípico aporta un $+1$ como cualquier otro punto mal predicho.

La tasa de aprendizaje se entiende con la misma analogía: el golfista podría intentar embocar de un solo golpe, pero se pasaría. El parámetro ν hace que cada corrección sea parcial, de manera que se avanza con pasos chicos y muchos modelos, y de ahí el compromiso entre la tasa de aprendizaje y la cantidad de iteraciones, que obliga a validarlas juntas.

El contraste final con AdaBoost se dice en el mismo lenguaje. AdaBoost no le indica al siguiente modelo cuánto falta sino a quién mirar, porque sube el peso de las observaciones que se erraron; Gradient Boosting le indica cuánto falta y en qué dirección. La consecuencia práctica es que AdaBoost queda atado a su pérdida exponencial mientras que Gradient Boosting acepta cualquier pérdida diferenciable, y por eso las implementaciones que hoy se usan en la práctica, como XGBoost, son variantes del segundo y no del primero (Chen & Guestrin, 2016).

Fuentes del Ejercicio 8. Material de cátedra (Volpacchio, s.f.): presentación `Ensemble Methods Universidad Austral` (esquema de boosting, taxonomía de perturbar y combinar, arcing, ponderación α_t y ejemplo numérico de las tres rondas) y documentos `Gradient boosting explicado` 1 a 3 (clase 6: modelado aditivo, analogía del golfista, ejemplo de los alquileres, vector de signos y la demostración de que el gradiente de la pérdida cuadrática es el vector residual). Freund y Schapire (1997) para la formulación original de AdaBoost, Friedman (2001) para Gradient Boosting y la recomendación sobre la tasa de aprendizaje, y Chen y Guestrin (2016) para las implementaciones actuales. El recálculo de los α_t es propio.

Ejercicio 9 · Random Forest

Random Forest. Describir el algoritmo y su diferencia con el resto de los algoritmos de ensamblaje. Describa los principales usos del algoritmo.

El algoritmo

Cada árbol de un Random Forest se entrena sobre una muestra bootstrap: de las N filas de entrenamiento se sortean N casos al azar con reemplazo. Sobre esa muestra el árbol crece con subespacio aleatorio de variables (Ho, 1995), es decir que de las M disponibles se fija un número $m \ll M$ tal que en cada nodo se sortean m y se divide por la mejor de ellas, con m constante durante todo el crecimiento del bosque. Y crece sin poda: cada árbol se desarrolla lo más posible y, con el tamaño mínimo de hoja en uno, cada registro termina en su propio nodo final.

Para predecir se evalúa la observación en cada árbol y se agregan las salidas, pero la forma de agregar depende de la tarea. En clasificación cada árbol vota y gana la clase más votada; en regresión se promedian las predicciones, y el criterio de división también cambia, porque en lugar de minimizar la impureza de Gini se minimiza la suma de cuadrados dentro de los nodos hijos. El documento trata las dos variantes en conjunto, pero la distinción importa.

El resultado teórico central es que la tasa de error del bosque queda acotada por $PE^* \leq \bar{\rho}(1-s^2)/s^2$, donde $\bar{\rho}$ es la correlación media entre árboles y s la fuerza de cada árbol individual, entendida como su capacidad de clasificar bien (Breiman, 2001). Es el análogo de la cota por vectores soporte del Ejercicio 2: una cota de generalización que se lee de la propia solución, acá desde la correlación y la fuerza en lugar de la cantidad de vectores soporte. No se usa para calcular nada; sirve para leer de dónde viene el error, porque depende de dos cosas y de nada más: más correlación implica más error, y más fuerza implica menos. El detalle que convierte al segundo paso en el corazón del método es que reducir m reduce tanto la correlación como la fuerza, y aumentarlo aumenta las dos, con lo cual existe un rango intermedio óptimo, por lo general bastante amplio. El documento afirma que ese es el único parámetro ajustable al que Random Forest resulta algo sensible.

Ahí se ve también por qué el promedio baja la varianza sin mover el sesgo. Los árboles están idénticamente distribuidos, así que el promedio conserva la esperanza y el sesgo queda igual al de un árbol solo, que es bajo porque el árbol crece sin poda. Lo que sí cae es la varianza, y cae tanto más cuanto menos correlacionados estén los árboles, que es precisamente lo que compra el sorteo de variables por nodo.

Como el muestreo es con reemplazo, alrededor de un tercio de los casos queda fuera de la bolsa de cada árbol: son los casos *out-of-bag*. Esos casos se predicen en los árboles que no los vieron, y la proporción de veces que la clase más votada no coincide con la verdadera es la estimación del error OOB. La consecuencia práctica es que no hace falta un conjunto de prueba separado, con lo cual toda la muestra se usa para entrenar. El documento va más lejos y llama insesgada a esa estimación, y ahí corresponde frenar: una serie de pruebas empíricas no demuestra insesgadez. Janitza y Hornung (2018) encuentran que el error OOB puede sobrestimar el error real, y que la sobrestimación es mayor justo donde no se la esperaría: con clases balanceadas, pocas observaciones y muchas variables.

De la construcción se desprenden otras dos herramientas. La importancia de variables se mide de dos maneras. Por permutación, se cuentan los votos a la clase correcta sobre las filas *out-of-bag*, se permutan al azar los valores de una variable y se vuelve a predecir; la caída promediada sobre todos los árboles es su importancia, con una lógica transparente, porque si al desordenar una variable el

desempeño no cae, esa variable no aportaba nada. Por impureza de Gini, que es la probabilidad de clasificar mal una observación de un nodo si se le asignara una clase al azar según la distribución del propio nodo, se acumula la caída que produce cada corte; esa caída corresponde al promedio ponderado por el tamaño de los nodos hijos, lo cual no es un detalle, porque un hijo individual puede quedar más impuro que el padre y lo que nunca sube es el promedio. El documento considera las dos medidas muy consistentes entre sí, aunque Strobl et al. (2007) le encontraron a la de Gini un sesgo a favor de las variables con más categorías, con dos fuentes: la selección sesgada dentro de cada árbol y el muestreo con reemplazo. Cuando las dos medidas discrepan conviene la de permutación.

La otra herramienta es la matriz de proximidad, de tamaño $N \times N$, donde la proximidad entre dos filas aumenta cada vez que caen en el mismo nodo terminal; como $1 - \text{prox}$ son distancias al cuadrado en un espacio euclídeo, de ella cuelgan varios de los usos que se describen más abajo.

Diferencias con el resto de los algoritmos de ensamblaje

Frente a bagging (Breiman, 1996), que también promedia árboles construidos sobre muestras bootstrap pero usa todas las variables en cada división, la diferencia esencial es el subespacio aleatorio, aunque no la única: bagging está definido para cualquier aprendiz base y Random Forest solo para árboles, que además crecen sin poda. La razón del sorteo es concreta: en bagging, si existe una variable muy predictiva, todos los árboles la eligen para la primera división y terminan pareciéndose mucho entre sí, y como el beneficio de promediar depende de que los componentes estén poco correlacionados, esa similitud limita la mejora.

Frente a boosting, que se desarrolla en el Ejercicio 8, Boosting es intrínsecamente secuencial, porque cada modelo depende del anterior, mientras que los árboles de un bosque son independientes y se construyen en paralelo. También difieren en qué componente del error ataca cada familia: Random Forest parte de árboles profundos, de bajo sesgo y alta varianza, y los promedia, con lo cual ataca la varianza, mientras que boosting parte de modelos débiles, de alto sesgo y baja varianza, y reduce el sesgo de forma aditiva. Hay acá una tensión con la presentación de la clase 6, que agrupa a bagging y a arcing con el argumento de que los dos reducen principalmente la varianza, aunque arcing es boosting; las dos afirmaciones se concilian si se lee que la presentación clasifica por el mecanismo de perturbación y no por la componente del error, porque boosting también recorta varianza al agregar, pero lo que lo distingue es que baja el sesgo.

Y difieren en el comportamiento frente al sobreajuste: Breiman afirma que los Random Forest no se sobreajustan y que se pueden correr tantos árboles como se desee, lo cual vale para el número de árboles pero no para la capacidad del modelo, porque con etiquetas ruidosas o con hojas de un solo caso un bosque sobreajusta como cualquier otro. De ahí la diferencia más relevante en la práctica: la cantidad de árboles es acá un parámetro de costo computacional, y en boosting uno de capacidad que hay que validar.

Frente a los métodos de ensamblaje en general, la presentación de la clase 6 da tres razones por las que ensamblar sirve, y Random Forest explota las tres: la estadística, porque con pocos datos hay muchas hipótesis igualmente buenas y promediar reduce el riesgo de elegir la equivocada; la computacional, porque el aprendizaje es una búsqueda local y partir de muchos puntos iniciales reduce la chance de quedar en un óptimo local; y la representacional, porque la hipótesis verdadera puede no pertenecer al espacio de un modelo único y la combinación interpola entre las halladas.

La misma presentación remarca lo que delimita cuándo conviene: el factor clave es la inestabilidad del clasificador base, de manera que los árboles sin podar mejoran mucho al ser perturbados y combinados, mientras que k vecinos más cercanos es estable y no mejora con bagging. Quedan dos parientes cercanos fuera de esa comparación. Los extra-trees (Geurts et al., 2006) sortean además el punto de corte en lugar de buscar el mejor, con lo cual bajan todavía más la correlación al precio de árboles más débiles, lo cual, en la cota de arriba, es mover $\bar{\rho}$ y s en el mismo sentido otra vez. Y el stacking pertenece a otra rama, porque no perturba nada: entrena modelos de familias distintas y después un meta-modelo que aprende a combinarlos con pesos ajustados, mientras que acá se combinan copias del mismo tipo de modelo.

Principales usos

Breiman y Cutler enumeran los usos en una lista larga y plana. Acá se reordenan según de dónde sale cada uno, porque agrupados así se entiende por qué existen: casi ninguno es una función que alguien le agregó al algoritmo, sino un subproducto de cómo está construido. El uso principal es la predicción sobre datos tabulares, clasificación o regresión indistintamente; de las tres virtudes que reivindica el documento, escrito en 2001, dos siguen intactas, porque corre eficiente en bases grandes y tolera miles de variables sin preselección, mientras que la tercera, la de una precisión excelente entre los algoritmos disponibles, hay que leerla con la fecha puesta.

Del primer subproducto, la muestra *out-of-bag*, sale la estimación del error sin reservar datos, que es lo que vuelve barato buscar m. Del segundo, la importancia de variables, sale la selección de variables, con el procedimiento que propone el documento: correr el bosque con todas, quedarse con las más importantes y volver a correrlo solo con esas, útil cuando hay miles de variables y ningún criterio previo para descartarlas.

Del tercero, la matriz de proximidad, cuelgan cuatro usos que en la lista original aparecían sueltos: la imputación de faltantes, con pesos dados por la proximidad; la detección de atípicos, por proximidad baja respecto del resto de su clase; el clustering no supervisado junto con las vistas en baja dimensión que se obtienen al escalar la matriz, que es la respuesta alternativa a la del Ejercicio 7; y los prototipos, que describen la relación entre variables y clase mediante la mediana y los cuartiles de los vecinos más cercanos de cada clase. Queda fuera del esquema el manejo de clases desbalanceadas, que no es un subproducto sino un agregado posterior por ponderación del error.

Hay algo que el documento no menciona y que conviene decir. Frente a un árbol único la mejora en varianza es grande, pero se pierde la lectura del modelo: un bosque de quinientos árboles no se lee como una lista de reglas, y lo que queda es la importancia de variables, que indica cuáles pesan pero no en qué dirección ni con qué interacción. Cuando hay que justificar cada decisión una por una, un árbol podado o una regresión logística son todavía mejores respuestas.

Fuentes del Ejercicio 9. Material de cátedra (Volpacchio, s.f.), clase 6: documento `randon forest explicado.pdf`, traducción de Breiman y Cutler (crecimiento del árbol, correlación y fuerza, rol de m, error *out-of-bag*, importancia, proximidad y prototipos), y presentación `Ensemble Methods Universidad Austral` (razones para ensamblar, perturbar y combinar, e inestabilidad del clasificador base). Breiman (1996) para bagging y Breiman (2001) para la cota de correlación y fuerza; Ho (1995) para el subespacio aleatorio y Geurts et al. (2006) para los extra-trees; Janitza y Hornung (2018) para la sobrestimación del error *out-of-bag* y Strobl et al. (2007) para el sesgo de la importancia de Gini, los dos reparos que el documento de cátedra no hace.

Ejercicio 10 · Pipeline en Python

Puede elegir cualquiera de las 2 opciones (si desea puede hacer las 2. A los fines de la evaluación uno es suficiente).

Opción 1

Seleccione dos datasets públicos (uno de clasificación y uno de regresión) y construya un Pipeline completo en Python que compare el desempeño de cuatro modelos: Support Vector Machine, Random Forest, Gradient Boosting y Multi-Layer Perceptron.

El pipeline debe incluir: (1) preprocesamiento adecuado de los datos, (2) definición explícita del espacio de búsqueda de hiperparámetros para cada modelo, (3) una estrategia de optimización (GridSearch o RandomSearch), y (4) validación mediante k-fold cross-validation.

Finalmente, presente una comparación clara del rendimiento de cada modelo en cada tarea.

Se resuelve la Opción 1. Sus cuatro modelos son los de la materia y se discuten con la notación de clase.

Datos y esquema de validación

- **Clasificación:** *Breast Cancer Wisconsin (Diagnostic)* (Wolberg et al., 1993): 569 tumores descritos por 30 medidas de los núcleos celulares en una imagen digitalizada; 212 son malignos (37,3 %).
- **Regresión:** *California Housing* (Pace & Barry, 1997): 20.640 grupos censales, 8 predictores y el valor mediano de la vivienda (en 100.000 USD) como respuesta. Se usa una submuestra aleatoria de 8.000 filas, tomada antes de partir, porque en una prueba previa un SVR (SVM de regresión) con $C = 1.000$ y $\gamma = 1$ tardó 46,5 s por ajuste con la submuestra de 8.000 filas (5.120 por ajuste dentro de la CV) y 80,7 s con 12.000.

Un hiperparámetro es un valor que se fija antes del entrenamiento (la penalización C de SVM, la cantidad de árboles), a diferencia de los pesos que el algoritmo aprende. Para elegirlos sin contaminar la evaluación hay dos niveles de partición. El externo es un *hold-out*: el 20 % de los datos (114 casos en clasificación, con estratificación por clase, y 1.600 en regresión) se aparta al principio y no interviene en ninguna decisión. El interno es una validación cruzada *k-fold* con $k = 5$ sobre el resto: el entrenamiento se parte en cinco bloques, cada configuración se ajusta cinco veces y en cada vuelta se evalúa sobre el bloque excluido (estratificado en clasificación). La configuración ganadora se reentrena con todo el entrenamiento y se mide una sola vez en el *hold-out*.

Una primera corrida informaba el promedio de validación cruzada que había servido para elegir la configuración: es optimista, porque la ganadora es en parte la que tuvo suerte con los bloques, y ese sesgo puede ser del orden de las diferencias entre algoritmos (Cawley & Talbot, 2010). Por eso se informan la media y el desvío estándar (DE) de los cinco bloques y, aparte, el puntaje en *hold-out*. Semilla: 42.

Preprocesamiento y métricas

Preproceso y modelo se encadenan en un `Pipeline` de scikit-learn, que la validación cruzada reajusta completo en cada bloque. Así se evita la fuga de información: si el `StandardScaler` (media 0 y desvío 1 por variable) se ajustara con todos los datos, los casos de evaluación entrarían

en el cálculo y la métrica sería optimista. Se escala para SVM, cuyo kernel RBF depende de distancias euclídeas, y para el MLP, cuyo descenso por gradiente converge mal con variables de magnitudes dispares. En los árboles se probó con y sin escalador y el F1 fue idéntico (Random Forest 0,9552, Gradient Boosting 0,9703 en ambos casos): un corte por umbral no cambia ante una transformación monótona. Se lo dejó fuera.

En el MLP de clasificación se desactivó la parada temprana (*early stopping*, que corta el entrenamiento cuando deja de mejorar el error sobre una fracción interna de validación): con 364 casos de entrenamiento en cada vuelta, el 10 % interno son unas 36 filas. En una prueba con una capa de 64 neuronas y tasa de aprendizaje 0,0003, la parada temprana bajó el F1 de 0,964 a 0,863. Sin ella no hubo avisos de falta de convergencia, y la red elegida convergió en la época 407 (tope: 2.000). En regresión, con unas 512 filas internas, se mantuvo para acotar el tiempo.

La clase positiva es maligno (scikit-learn la codifica como 0; se recodificó). Se elige por el F1 de esa clase: la media armónica entre la precisión (fracción de verdaderos malignos entre los predichos como malignos) y la exhaustividad (fracción de los malignos reales que se detectan). Una primera corrida usaba el F1 ponderado con el argumento del desbalance, y el argumento era erróneo: ese promedio pesa cada clase por su tamaño y favorece a la benigna, la mayoritaria, cuando el error costoso es un maligno no detectado. Se informan también la exactitud (proporción de aciertos) y el AUC, el área bajo la curva ROC: la probabilidad de que un maligno al azar reciba un puntaje mayor que un benigno al azar. En regresión se elige por RMSE, la raíz del error cuadrático medio, en unidades de la respuesta, y se informa el R^2, la fracción de varianza explicada.

Estrategia de optimización y validación cruzada

La búsqueda es `RandomizedSearchCV`, que evalúa una muestra aleatoria de la grilla en lugar de todas sus combinaciones (lo que hace `GridSearchCV`): con el mismo presupuesto prueba más valores distintos de cada hiperparámetro (Bergstra & Bengio, 2012). Se evaluaron 30 combinaciones por modelo; cuando la grilla tenía 30 o menos, la búsqueda la recorrió completa, como un `GridSearchCV` (SVM en clasificación, Random Forest en clasificación, Random Forest en regresión).

Espacio de búsqueda de hiperparámetros

La Tabla 7 muestra el espacio de cada modelo en cada tarea. Las grillas son logarítmicas en los parámetros de escala, como recomiendan Hsu et al. (2003) para SVM. En SVM, C es el costo por violar el margen y γ es el inverso del ancho del kernel RBF: con la dispersión σ del Ejercicio 2, $\gamma = 1/(2\sigma^2)$, y un γ grande da una frontera muy local; en regresión se suma ε, el radio del tubo de insensibilidad (Volpacchio, s.f.). En Random Forest se fijaron 300 árboles, porque agregar árboles no sobreajusta (Breiman, 2001), y se buscó sobre m, las variables sorteadas en cada corte, que mueve la fuerza de cada árbol (su acierto) y la correlación entre árboles, y sobre el tamaño de hoja. En Gradient Boosting cada árbol se ajusta a los pseudo-residuos, el gradiente negativo de la pérdida respecto de la predicción actual (los residuos, con pérdida cuadrática); se combinan la tasa ν del Ejercicio 8, que encoge cada paso, las etapas y la profundidad (Volpacchio, s.f.). En el MLP, α penaliza la suma de los pesos al cuadrado y Adam es el optimizador.

Tabla 7: Espacio de búsqueda por modelo y tarea; en negrita, el valor elegido.

Modelo	Hiperparámetro	Clasificación	Regresión
SVM	C	0,01; 0,1; 1; **10**; 100; 1.000	0,1; **1**; 10; 100
	γ	0,0001; 0,001; **0,01**; 0,1; 1	0,01; 0,03; 0,1; **0,3**; 1
	ε		0,05; **0,1**; 0,2; 0,4; 0,8
	fijos	kernel rbf	kernel rbf
Random Forest	m (max_features)	2; 4; 6; 10; **15**; 30	1; 2; **3**; 4; 6; 8
	hoja mínima	**1**; 2; 4; 8	**1**; 2; 4; 8; 16
	fijos	árboles 300	árboles 300
Gradient Boosting	etapas	25; 50; 100; **200**; 400	100; 200; 400; 800; **1.600**
	ν (learning_rate)	0,01; 0,03; 0,1; **0,3**; 1	0,01; **0,03**; 0,1; 0,3
	profundidad	**1**; 2; 3; 4; 6	2; 3; 4; **6**; 8
MLP	capas ocultas	(16); **(32)**; (64); (128); (64, 32)	(32); (64); (128); **(64, 32)**; (128, 64)
	α	0,00001; 0,0001; 0,001; **0,01**; 0,1; 1	0,00001; 0,0001; **0,001**; 0,01; 0,1; 1
	tasa inicial	0,0001; **0,0003**; 0,001; 0,003; 0,01; 0,03	0,0003; 0,001; 0,003; **0,01**; 0,03
	fijos	épocas máx. 2.000; parada temprana no	épocas máx. 2.000; parada temprana sí; paciencia 20

Nota. Elaboración propia. Hoja mínima: observaciones por hoja; tasa inicial: paso inicial del optimizador (Adam); paciencia: épocas sin mejora antes de la parada temprana.

Resultados

Tabla 8: Clasificación en Breast Cancer Wisconsin: CV interna y hold-out.

Modelo	F1 CV (media ± DE)	F1 test	Exactitud test	AUC test	FN test	FP test	Tiempo (s)
SVM	0,967 ± 0,016	0,976	0,982	0,996	2	0	1,6
Random Forest	0,955 ± 0,025	0,963	0,974	0,993	3	0	7,0
Gradient Boosting	0,970 ± 0,016	0,950	0,965	0,989	4	0	5,9
MLP	0,973 ± 0,017	0,988	0,991	0,997	1	0	5,1

Nota. Elaboración propia con datos de Wolberg et al. (1993). Hold-out de 114 casos (42 malignos). FN: malignos predichos como benignos; FP: a la inversa; tiempo: duración de la búsqueda.

Tabla 9: Regresión en California Housing: CV interna y hold-out.

Modelo	RMSE CV (media ± DE)	RMSE test	R^2 CV	R^2 test	Tiempo (s)
SVM	0,561 ± 0,012	0,530	0,763	0,784	39,5
Random Forest	0,533 ± 0,009	0,487	0,786	0,818	66,2
Gradient Boosting	0,500 ± 0,013	0,457	0,812	0,839	172,4
MLP	0,554 ± 0,014	0,600	0,769	0,723	30,8

Nota. Elaboración propia con datos de Pace y Barry (1997), submuestra de 8.000 filas; RMSE en 100.000 USD.

Comparación de rendimiento

En clasificación la media más alta de validación cruzada es la de MLP (0,973 ± 0,017), y la distancia entre el primero y el último, 0,018, no supera el mayor DE (0,025): con este criterio

informal, los cuatro modelos no se distinguen. En *hold-out* difieren en entre 1 y 4 malignos no detectados sobre 42 (ningún modelo produjo falsos positivos), y cada uno resta 0,012 de F1, del orden del DE entre bloques. El empate es del conjunto antes que de los modelos: las clases son casi linealmente separables en el espacio estandarizado, como mide el alineamiento del Ejercicio 5, y ahí los cuatro llegan al mismo techo. SVM eligió $\gamma = 0{,}01$, es decir $\sigma = 1/\sqrt{2\gamma} \approx 7{,}1$, del orden de la distancia típica entre dos casos estandarizados ($\sqrt{2p} \approx 7{,}7$): un kernel ancho a esa escala (inferencia propia). Gradient Boosting eligió profundidad 1: cada árbol usa una variable, así que el modelo es una suma de efectos por variable, sin interacciones.

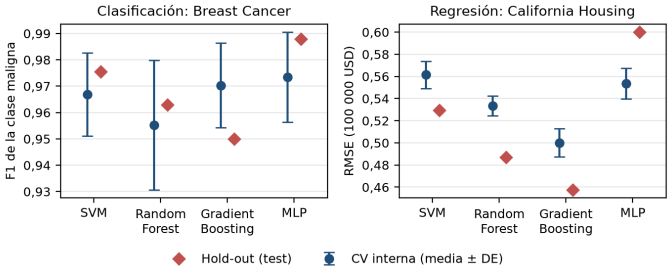


Figura 3: Métrica principal por modelo: media ± DE de la CV interna y puntaje en hold-out.

Nota. Elaboración propia con la corrida de `codigo/e10_pipeline/pipeline_opcion1.py`. En clasificación, mayor es mejor; en regresión, menor es mejor.

En regresión, a diferencia de la clasificación, la comparación sí separa. Gana Gradient Boosting, con RMSE de 0,500 ± 0,013 en validación cruzada y 0,457 en *hold-out* (unos 45.700 USD de error típico por grupo censal; $R^2 = 0{,}839$). Su ventaja sobre Random Forest, 0,033, es 2,6 veces el mayor de los dos DE, y los dos ensambles (modelos que combinan muchos árboles) superan a SVM y al MLP en las dos particiones, como los árboles frente a las redes en Grinsztajn et al. (2022), que no incluyen SVM. Una explicación posible (inferencia propia) es que el valor cambia en forma brusca con la ubicación (latitud y longitud son predictores), algo que un árbol capta con pocos cortes y que el kernel RBF y el MLP, funciones suaves, aproximan peor. Que Gradient Boosting supere a Random Forest es coherente con su mecanismo: cada etapa corrige los pseudo-residuos de las anteriores y reduce el sesgo; el promedio del bosque reduce sobre todo la varianza.

La brecha entre validación cruzada y *hold-out* no tiene un signo único. En SVM y en los árboles el RMSE de test es menor: el modelo final se reentrena con 6.400 filas y no con 5.120, y el *hold-out* es una sola partición. El MLP empeora de 0,554 a 0,600; se conjetura, sin verificarlo, que una red con una sola inicialización es más inestable que un promedio de árboles.

La primera corrida dejaba C y ε del SVR en el máximo de su grilla; tras ampliar las grillas quedan 4 valores en un borde. De ellos, 3 no se pueden extender: hoja mínima 1 en Random Forest, en las dos tareas (árboles completos, como en Breiman, 2001), y profundidad 1 en Gradient Boosting de clasificación. El restante es una limitación: Gradient Boosting de regresión eligió 1.600 etapas, el máximo, con $\nu = 0{,}03$ (con una tasa chica hacen falta más etapas); más etapas podrían bajar algo el RMSE, pero esa búsqueda ya es la más costosa (172 s de 331 s) y su costo crece con las etapas.

Fuentes del Ejercicio 10. Material de cátedra (Volpacchio, s.f.): presentaciones de SVM y de ensambles y apunte de redes. Wolberg et al. (1993) y Pace y Barry (1997) para los datos; Bergstra y Bengio (2012) para la búsqueda aleatoria; Hsu et al. (2003) para las grillas logarítmicas; Cawley y Talbot (2010) para el sesgo de selección; Breiman (2001) para el número de árboles; Grinsztajn et al. (2022) para la comparación entre árboles y redes. Código propio: Apéndice A.7.

Apéndice · Código

El código se desarrolló y ejecutó fuera del `.Rmd`. Se incluyen los scripts de los Ejercicios 1 (LSTM y figuras), 4, 6, 7 y 10, completos y en letra chica; no forman parte del desarrollo de cada ejercicio.

A.1 LSTM serie temporal (Ejercicio 1.VI)

```
"""
Ejercicio 1.VI: LSTM sobre una serie sintética (TensorFlow/Keras).

Ventana de 20, corte temporal 80/20, shuffle=False, cuatro configuraciones,
tres semillas por configuración. La elección se hace por validación; el test
se mira una sola vez al final.

Correr desde la raíz del repo:
  .venv-tf/bin/python codigo/e1_lstm/lstm_serie.py
"""

from __future__ import annotations

import json
import time
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from matplotlib.ticker import FuncFormatter

SEED = 42
# Con una sola semilla, el orden entre configuraciones podía ser azar de la
# inicialización (lstm_32 y lstm_64 quedaban a pocas milésimas de MAE). Tres
# semillas dan una idea de esa variación sin multiplicar el tiempo de corrida.
SEEDS = [42, 43, 44]
# La ventana de 20 cubre cerca del 40 % de un ciclo del seno rápido (período
# 2*pi/0,12  52): suficiente para estimar pendiente y curvatura locales.
LOOKBACK = 20
EPOCHS = 20
BATCH = 32
NOISE_SD = 0.1
OUT = Path(__file__).resolve().parents[1] / "resultados" / "e1"
OUT.mkdir(parents=True, exist_ok=True)


def lstm_param_count(units: int, input_dim: int) -> int:
    # Fórmula del apunte: 4 bloques (3 puertas + candidata), cada uno con
    # pesos de entrada, pesos recurrentes y sesgo.
    return 4 * ((units * input_dim) + units * units + units)


def params_stack(layers: list[int], input_dim: int = 1) -> int:
    # En una LSTM apilada, la entrada de la segunda capa es la salida
    # de la primera: input_dim pasa a ser las units de la capa anterior.
    total, prev = 0, input_dim
    for units in layers:
        total += lstm_param_count(units, prev)
        prev = units
    return total
```

```
def clean_signal(n: int) -> np.ndarray:
    t = np.arange(n)
    return np.sin(0.05 * t) + 0.3 * np.sin(0.12 * t)


def make_series(n: int = 1200) -> np.ndarray:
    # Serie arbitraria: dos senos de distinta frecuencia más ruido gaussiano.
    # Al conocer la señal limpia se puede calcular el piso de error que
    # ningún modelo puede bajar (el ruido no es predecible).
    rng = np.random.default_rng(SEED)
    return clean_signal(n) + NOISE_SD * rng.normal(size=n)


def windowing(series: np.ndarray, lookback: int = LOOKBACK):
    X, y = [], []
    for i in range(lookback, len(series)):
        X.append(series[i - lookback : i])
        y.append(series[i])
    return np.asarray(X)[..., np.newaxis], np.asarray(y)


def build_model(layers_spec: list[int], lookback: int):
    from tensorflow import keras
    from tensorflow.keras import layers

    model = keras.Sequential()
    model.add(layers.Input(shape=(lookback, 1)))
    for i, units in enumerate(layers_spec):
        # Solo la última LSTM devuelve un vector; las anteriores pasan la
        # secuencia completa a la siguiente.
        model.add(layers.LSTM(units, return_sequences=(i < len(layers_spec) - 1)))
    model.add(layers.Dense(1))
    model.compile(optimizer="adam", loss="mse", metrics=["mae"])
    return model


def main() -> None:
    import tensorflow as tf
    from tensorflow import keras

    t0 = time.time()
    series = make_series()
    # No se escala: la serie ya vive en [-1,5; 1,5], el rango donde tanh y la
    # sigmoidal de las puertas no saturan.
    X, y = windowing(series)
    split = int(0.8 * len(X))
    X_train, X_test = X[:split], X[split:]
    y_train, y_test = y[:split], y[split:]
    # 10 % final del train como validación temporal: es el tramo que se usa
    # para elegir configuración. El test queda para el número final.
    v = int(0.9 * len(X_train))
    X_tr, X_val = X_train[:v], X_train[v:]
    y_tr, y_val = y_train[:v], y_train[v:]

    # Dos referencias para leer el MAE de la LSTM:
    # - persistencia: predecir y_t con y_{t-1}, el modelo más tonto posible;
```

```
# - piso de ruido: MAE contra la señal limpia, lo mínimo alcanzable.
persist_mae = float(np.mean(np.abs(y_test - X_test[:, -1, 0])))
clean_test = clean_signal(len(series))[LOOKBACK:][split:]
floor_mae = float(np.mean(np.abs(y_test - clean_test)))

configs = [
    ("lstm_32", [32]),
    ("lstm_64", [64]),
    ("lstm_32_16", [32, 16]),
    ("lstm_64_32", [64, 32]),
]

runs = []
models = {}
for name, layers_spec in configs:
    for seed in SEEDS:
        tf.keras.utils.set_random_seed(seed)
        model = build_model(layers_spec, LOOKBACK)
        # shuffle=False: el orden temporal de los lotes se respeta.
        hist = model.fit(
            X_tr,
            y_tr,
            epochs=EPOCHS,
            batch_size=BATCH,
            validation_data=(X_val, y_val),
            shuffle=False,
            verbose=0,
        )
        test_mse, test_mae = model.evaluate(X_test, y_test, verbose=0)
        runs.append(
            {
                "config": name,
                "seed": seed,
                "val_mae": float(hist.history["val_mae"][-1]),
                "test_mse": float(test_mse),
                "test_mae": float(test_mae),
            }
        )
        print(runs[-1])
        if seed == SEED:
            models[name] = model

df_runs = pd.DataFrame(runs)
df_runs.to_csv(OUT / "lstm_runs.csv", index=False)

rows = []
for name, layers_spec in configs:
    r = df_runs[df_runs["config"] == name]
    rows.append(
        {
            "config": name,
            "layers": "-".join(str(u) for u in layers_spec),
            "params_formula_lstm": params_stack(layers_spec),
            "params_keras": int(models[name].count_params()),
            "val_mae_mean": r["val_mae"].mean(),
            "test_mae_mean": r["test_mae"].mean(),
            "test_mae_sd": r["test_mae"].std(ddof=1),
            "test_mse_mean": r["test_mse"].mean(),
```

```
            }
        )
    df = pd.DataFrame(rows)
    df.to_csv(OUT / "lstm_configs.csv", index=False)

    # Se elige por validación. Elegir por test sería usar el test para decidir
    # y el número final dejaría de ser una estimación honesta.
    best = df.loc[df["val_mae_mean"].idxmin()]
    best_name = best["config"]
    best_model = models[best_name]

    preds = best_model.predict(X_test, verbose=0).ravel()
    fig, ax = plt.subplots(figsize=(7, 2.3))
    ax.plot(y_test, label="real", lw=1.0)
    ax.plot(preds, label=f"predicción {best_name}", lw=1.1)
    ax.legend(loc="upper left", fontsize=8)
    ax.set_xlabel("índice temporal (test)")
    ax.set_ylabel("valor")
    ax.yaxis.set_major_formatter(FuncFormatter(lambda v, _: f"{v:.1f}".replace(".", ",")))
    fig.tight_layout()
    fig.savefig(OUT / "lstm_pred.png", dpi=150)
    plt.close(fig)

    with open(OUT / "keras_summary.txt", "w", encoding="utf-8") as f:
        best_model.summary(print_fn=lambda line, **_: f.write(line + "\n"))

    n_fit = len(X_tr)
    meta = {
        "status": "ok",
        "backend": "tensorflow-cpu",
        "tf_version": tf.__version__,
        "keras_version": keras.__version__,
        "seeds": SEEDS,
        "lookback": LOOKBACK,
        "epochs": EPOCHS,
        "split": "temporal_80_20",
        "shuffle": False,
        "n_fit": n_fit,
        "n_val": len(X_val),
        "n_test": len(X_test),
        "noise_sd": NOISE_SD,
        "persist_mae": persist_mae,
        "floor_mae": floor_mae,
        "best_by_val": best_name,
        "d_over_l_best": float(best["params_keras"]) / n_fit,
        "runtime_s": round(time.time() - t0, 1),
        "formula": "4*((units*input_dim)+units^2+units)",
    }
    (OUT / "meta.json").write_text(json.dumps(meta, indent=2, ensure_ascii=False))
    print("Escrito en", OUT)


if __name__ == "__main__":
    main()
```

A.2 Figuras CNN y celda LSTM (Ejercicio 1)

Los diagramas de `codigo/resultados/e1/cnn_capas.png` y `lstm_celda.png` los generó `codigo/e1_lstm/figuras.py`.

```
"""
Figuras del Ejercicio 1: capas CNN y celda LSTM.

Salida: codigo/resultados/e1/cnn_capas.png y lstm_celda.png
Correr:  .venv-tf/bin/python codigo/e1_lstm/figuras.py
"""

from __future__ import annotations

from pathlib import Path

import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch

OUT = Path(__file__).resolve().parents[1] / "resultados" / "e1"
OUT.mkdir(parents=True, exist_ok=True)


def _box(ax, x, y, w, h, text, fc="#f4f4f4", ec="#222222", size=8, weight="normal"):
    p = FancyBboxPatch(
        (x, y),
        w,
        h,
        boxstyle="round,pad=0.02,rounding_size=0.08",
        linewidth=1.1,
        facecolor=fc,
        edgecolor=ec,
    )
    ax.add_patch(p)
    ax.text(
        x + w / 2,
        y + h / 2,
        text,
        ha="center",
        va="center",
        fontsize=size,
        color="#111111",
        fontweight=weight,
        wrap=True,
    )


def _arrow(ax, x1, y1, x2, y2):
    ax.annotate(
        "",
        xy=(x2, y2),
        xytext=(x1, y1),
        arrowprops=dict(arrowstyle="-|>", color="#222222", lw=1.0),
    )


def figura_cnn() -> None:
    # Lienzo chato y tipografía grande: en el PDF entra al ancho del texto
    # y los rótulos tienen que leerse a unos 9 pt, no como una tira.
    fig, ax = plt.subplots(figsize=(7.4, 1.58))
    ax.set_xlim(0, 17.6)
    ax.set_ylim(0, 2.05)
    ax.axis("off")
```

```
    boxes = [
        (0.10, 0.46, 1.9, 1.22, "Imagen\nn × n × 3", "#e8e8e8"),
        (2.22, 0.46, 2.15, 1.22, "CONV\n+ ReLU", "#d9e2ec"),
        (4.59, 0.46, 1.7, 1.22, "POOL\nmáximo", "#f0e6d2"),
        (6.51, 0.46, 2.25, 1.22, "CONV 2\n+ ReLU", "#d9e2ec"),
        (8.98, 0.46, 1.7, 1.22, "POOL\nmáximo", "#f0e6d2"),
        (10.90, 0.46, 1.85, 1.22, "Aplanado\nvector", "#e8e8e8"),
        (12.97, 0.46, 1.85, 1.22, "Capa\ndensa", "#dce8d4"),
        (15.04, 0.46, 2.15, 1.22, "Softmax\nclases", "#dce8d4"),
    ]
    for x, y, w, h, t, fc in boxes:
        _box(ax, x, y, w, h, t, fc=fc, size=9.0)

    xs = [b[0] for b in boxes]
    ws = [b[2] for b in boxes]
    for i in range(len(xs) - 1):
        _arrow(ax, xs[i] + ws[i] + 0.02, 1.07, xs[i + 1] - 0.02, 1.07)

    ax.text(
        8.8,
        0.08,
        "El bloque CONV-POOL se puede repetir. Cada filtro comparte sus pesos.",
        ha="center",
        fontsize=8,
        color="#333333",
    )
    fig.tight_layout()
    fig.savefig(OUT / "cnn_capas.png", dpi=180, bbox_inches="tight", facecolor="white")
    plt.close(fig)


def figura_lstm() -> None:
    # Las cuatro puertas van en una fila. Cada flecha baja a su caja y no
    # atraviesa otra: olvido, entrada y candidata entran a c_t; la salida, a h_t.
    # Con fuentes de 9 pt, al 50 % del ancho las ecuaciones quedaban en ~4 pt
    # impresas: se suben las fuentes para que se lean al 60 %.
    fig, ax = plt.subplots(figsize=(7.05, 2.32))
    ax.set_xlim(0, 13.8)
    ax.set_ylim(0, 4.45)
    ax.axis("off")

    ax.text(6.9, 4.18, "Celda LSTM, paso $t$", ha="center", fontsize=13, fontweight="bold")

    _box(ax, 1.85, 2.58, 2.45, 1.12, "Olvido $f_t$\nsigmoidal", fc="#f7d6d6", size=12)
    _box(ax, 4.55, 2.58, 2.45, 1.12, "Entrada $i_t$\nsigmoidal", fc="#d6e8f7", size=12)
    _box(ax, 7.25, 2.58, 2.55, 1.12, "Candidata $\\tilde{c}_t$\ntanh", fc="#e8e0d6", size=12)
    _box(ax, 10.15, 2.58, 2.45, 1.12, "Salida $o_t$\nsigmoidal", fc="#d6f0d6", size=12)

    _box(
        ax,
        1.85,
        0.38,
        7.95,
        1.22,
        r"$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$",
        fc="#fff6cc",
        size=13,
```

```
    )
    _box(ax, 10.15, 0.72, 2.45, 0.88, r"$h_t = o_t \odot \tanh(c_t)$",
         fc="#e8e8e8", size=10)

    # De cada puerta a la caja de abajo. Terminan en el borde superior.
    _arrow(ax, 3.07, 2.58, 3.07, 1.62)
    _arrow(ax, 5.77, 2.58, 5.77, 1.62)
    _arrow(ax, 8.52, 2.58, 8.52, 1.62)
    _arrow(ax, 11.37, 2.58, 11.37, 1.62)

    # c_t alimenta a h_t por el costado, sin cruzar la caja gris.
    _arrow(ax, 9.82, 1.15, 10.13, 1.15)

    ax.text(0.0, 3.05, r"$x_t,\, h_{t-1}$", fontsize=12)
    _arrow(ax, 1.42, 3.14, 1.83, 3.14)
    ax.text(0.15, 0.85, r"$c_{t-1}$", fontsize=13)
    _arrow(ax, 1.15, 0.98, 1.83, 0.98)

    _arrow(ax, 12.62, 1.16, 13.15, 1.16)
    ax.text(13.2, 1.02, r"$h_t$", fontsize=13)
    # c_t sale por debajo de h_t: la caja gris queda arriba de esta flecha.
    _arrow(ax, 9.82, 0.52, 13.15, 0.52)
    ax.text(13.2, 0.38, r"$c_t$", fontsize=13)
    fig.tight_layout()
    fig.savefig(OUT / "lstm_celda.png", dpi=180, bbox_inches="tight", facecolor="white")
    plt.close(fig)


if __name__ == "__main__":
    figura_cnn()
    figura_lstm()
    print("Figuras en", OUT)
```

A.3 AG binario y AG real sobre Rastrigin (Ejercicio 4)

```
"""Ejercicio 4: AG binario contra AG real sobre la función de Rastrigin, con DEAP.

Se eligió Rastrigin porque tiene un mínimo global en el origen rodeado de una grilla
de mínimos locales: con la esfera x1^2 + x2^2 cualquier método de gradiente llega solo,
y el AG no mostraba nada. Los dos AG comparten población, generaciones, selección y
probabilidades; solo cambian la representación y los operadores, que es la diferencia
que pide el enunciado.

Salidas en codigo/resultados/e4/: ag_corridas.csv (una fila por semilla y codificación)
y meta.json. Se armó también una figura de convergencia, pero las bandas
intercuartílicas de las dos codificaciones se pisaban de punta a punta; la tabla del
informe (óptimo, cuenca, valle vecino) dice más en menos lugar, y la figura se sacó.
"""

import json
import random
from pathlib import Path

import numpy as np
import pandas as pd
from deap import algorithms, base, creator, tools

SEED = 42
```

```
SEMILLAS = list(range(SEED, SEED + 20))  # 20 corridas: con una sola semilla el azar decide
N_VAR = 2
LIM = 5.12  # dominio clásico de Rastrigin
BITS = 16  # por variable: paso de 10,24 / (2^16 - 1), cerca de 1,6e-4
POP, NGEN, CXPB, MUTPB = 50, 100, 0.8, 0.2
UMBRAL = 1e-2  # se cuenta como "llegó al óptimo global" si f < 0,01

OUT = Path(__file__).resolve().parents[1] / "resultados" / "e4"
OUT.mkdir(parents=True, exist_ok=True)


def rastrigin(x):
    x = np.asarray(x, dtype=float)
    return float(10 * len(x) + np.sum(x**2 - 10 * np.cos(2 * np.pi * x)))


def decodificar(bits):
    # x = a + decimal(cadena) * (b - a) / (2^m - 1), la fórmula del apunte
    xs = []
    for k in range(N_VAR):
        trozo = bits[k * BITS:(k + 1) * BITS]
        entero = int("".join(str(b) for b in trozo), 2)
        xs.append(-LIM + entero * (2 * LIM) / (2**BITS - 1))
    return xs


# weights=(-1.0,) porque se minimiza
creator.create("FitnessMin", base.Fitness, weights=(-1.0,))
creator.create("Individuo", list, fitness=creator.FitnessMin)


def toolbox_binario():
    tb = base.Toolbox()
    tb.register("gen", random.randint, 0, 1)
    tb.register("individuo", tools.initRepeat, creator.Individuo, tb.gen, N_VAR * BITS)
    tb.register("poblacion", tools.initRepeat, list, tb.individuo)
    tb.register("evaluate", lambda ind: (rastrigin(decodificar(ind)),))
    tb.register("mate", tools.cxOnePoint)  # corta y cambia colas, como en las diapositivas
    tb.register("mutate", tools.mutFlipBit, indpb=1.0 / (N_VAR * BITS))
    tb.register("select", tools.selTournament, tournsize=3)
    return tb


def recortar(ind):
    # cxBlend y la mutación gaussiana pueden sacar el gen del dominio: se lo lleva al borde
    for i, v in enumerate(ind):
        ind[i] = min(max(v, -LIM), LIM)
    return ind


def toolbox_real():
    tb = base.Toolbox()
    tb.register("gen", random.uniform, -LIM, LIM)
    tb.register("individuo", tools.initRepeat, creator.Individuo, tb.gen, N_VAR)
    tb.register("poblacion", tools.initRepeat, list, tb.individuo)
    tb.register("evaluate", lambda ind: (rastrigin(ind),))

    def cruce(a, b):
```

```
        tools.cxBlend(a, b, alpha=0.5)
        return recortar(a), recortar(b)

    def mutacion(ind):
        # sigma 0,3: primero se probó 1,0 y el AG saltaba entre valles sin afinar
        tools.mutGaussian(ind, mu=0.0, sigma=0.3, indpb=0.5)
        return (recortar(ind),)

    tb.register("mate", cruce)
    tb.register("mutate", mutacion)
    tb.register("select", tools.selTournament, tournsize=3)
    return tb


def correr(nombre, tb, semilla):
    random.seed(semilla)
    np.random.seed(semilla)
    pop = tb.poblacion(n=POP)
    hof = tools.HallOfFame(1)  # guarda el mejor visto, no reinyecta (sin elitismo)
    stats = tools.Statistics(lambda ind: ind.fitness.values[0])
    stats.register("min", np.min)
    _, log = algorithms.eaSimple(pop, tb, cxpb=CXPB, mutpb=MUTPB, ngen=NGEN,
                                 stats=stats, halloffame=hof, verbose=False)
    # mejor hasta la generación g (acumulado): sin elitismo el mínimo de P(t) puede subir
    mejor_acum = np.minimum.accumulate(log.select("min"))
    mejor = hof[0]
    x = decodificar(mejor) if nombre == "binario" else list(mejor)
    llego = np.where(mejor_acum < UMBRAL)[0]
    return {
        "codificacion": nombre,
        "semilla": semilla,
        "mejor_f": float(mejor.fitness.values[0]),
        "x1": x[0],
        "x2": x[1],
        "gen_umbral": int(llego[0]) if len(llego) else None,
    }


filas = []
for nombre, fabrica in [("binario", toolbox_binario), ("real", toolbox_real)]:
    for s in SEMILLAS:
        filas.append(correr(nombre, fabrica(), s))

corridas = pd.DataFrame(filas)
corridas.to_csv(OUT / "ag_corridas.csv", index=False)


def resumen(nombre):
    d = corridas[corridas.codificacion == nombre]
    return {
        "mediana_final": float(d.mejor_f.median()),
        "n_umbral": int((d.mejor_f < UMBRAL).sum()),
        "gen_umbral_mediana": float(d.gen_umbral.dropna().median()),
        # cuenca global: |x_i| < 0,5 en las dos variables, aunque falte el ajuste fino.
        # Se agregó al ver que varias corridas reales terminaban a pocas centésimas del
        # origen con f > 0,01, mientras las binarias fallidas quedaban clavadas en x = ±1.
        "n_cuenca": int(((d.x1.abs() < 0.5) & (d.x2.abs() < 0.5)).sum()),
        "n_vecino": int(((d.x1.abs() - 1).abs().lt(0.1)
```

```
                        | (d.x2.abs() - 1).abs().lt(0.1)).sum()),
    }

# Por qué las binarias fallidas no salen de x = 1: el entero que codifica x = 1 y los dos
# enteros que rodean a x = 0 difieren en muchos bits. Con indpb = 1/32 por bit, cambiar
# varios bits justos en la misma mutación es casi imposible (precipicio de Hamming).
def entero(x):
    return round((x + LIM) * (2**BITS - 1) / (2 * LIM))


k1 = entero(1.0)
centro = LIM * (2**BITS - 1) / (2 * LIM)  # x = 0 cae entre dos enteros
cero = [int(np.floor(centro)), int(np.ceil(centro))]
hamming_1_0 = min(bin(k1 ^ k0).count("1") for k0 in cero)


meta = {
    "seed": SEED, "n_semillas": len(SEMILLAS), "pop": POP, "ngen": NGEN,
    "cxpb": CXPB, "mutpb": MUTPB, "bits": BITS, "lim": LIM, "umbral": UMBRAL,
    "paso_binario": 2 * LIM / (2**BITS - 1), "n_evals": POP * (NGEN + 1),
    "binario": resumen("binario"), "real": resumen("real"),
    "hamming_1_0": hamming_1_0, "p_mut_bit": 1.0 / (N_VAR * BITS),
}
(OUT / "meta.json").write_text(json.dumps(meta, indent=2, ensure_ascii=False))
print(json.dumps(meta, indent=2, ensure_ascii=False))
```

A.4 Alineamiento kernel-objetivo sobre Breast Cancer (Ejercicio 5)

Produce `codigo/resultados/e5/alineamiento.csv`: el alineamiento de cada kernel candidato sobre la partición de entrenamiento del Ejercicio 10.

```
"""
Ejercicio 5 - Alineamiento kernel-objetivo sobre un data set real.

La pregunta del enunciado es si existen metodos sencillos para elegir el kernel
apropiado en un determinado data set. El Ejercicio 5 contesta con bibliografia,
y hasta aca era el unico ejercicio del examen sin una medicion propia. Esto lo
corrige: se calcula el alineamiento de Cristianini et al. (2002) sobre los
mismos datos y la misma particion del Ejercicio 10, y se compara el orden que
produce con el que produjo la validacion cruzada.

Eso es lo que de verdad se le pide al metodo. El alineamiento se vende como un
atajo: elegir kernel sin entrenar un modelo por candidato. Si el atajo sirve,
tiene que coincidir con lo que dio entrenar. Si no coincide, tambien es un
resultado, y es el que hay que informar.

    A = <K, yy'>_F / (||K||_F * ||yy'||_F)

con y en {-1, +1}. Se usa la version SIN centrar, que es la del paper original
y la que esta escrita en el cuerpo del examen. La variante centrada es
posterior; no se mezcla para que el numero del informe sea el de la formula
que el informe muestra.

Salida: codigo/resultados/e5/alineamiento.csv
Correr:  python codigo/e5_kernels/alineamiento_kernel.py
"""

import csv
```

```
import json
import os

import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.metrics.pairwise import linear_kernel, polynomial_kernel, rbf_kernel
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

AQUI = os.path.dirname(os.path.abspath(__file__))
SAL = os.path.join(AQUI, "..", "resultados", "e5")
os.makedirs(SAL, exist_ok=True)

# Los mismos del Ejercicio 10. Si cambian alla, cambian aca, porque el valor de
# esta comparacion es que las dos miren exactamente el mismo conjunto.
SEED = 42
TEST_SIZE = 0.2

datos = load_breast_cancer()
X, y01 = datos.data, datos.target

# El Ejercicio 10 recodifica: maligno es la clase positiva. scikit-learn la trae
# como 0, asi que se da vuelta antes de armar el vector de etiquetas.
y = np.where(y01 == 0, 1, -1)

Xtr, _, ytr, _ = train_test_split(X, y, test_size=TEST_SIZE, random_state=SEED,
                                  stratify=y)

# Se escala con el mismo StandardScaler del pipeline y ajustado SOLO con el
# entrenamiento. Sin esto el alineamiento del RBF no significa nada: gamma
# opera sobre distancias euclideas y las 30 variables tienen escalas muy
# distintas (el area va a miles, la dimension fractal a centesimas).
Xtr = StandardScaler().fit_transform(Xtr)
n = Xtr.shape[0]

ideal = np.outer(ytr, ytr)
norma_ideal = np.linalg.norm(ideal)  # con y en {-1,+1} da exactamente n


def alineamiento(K):
    return float(np.sum(K * ideal) / (np.linalg.norm(K) * norma_ideal))


# Los candidatos son los del enunciado del Ejercicio 5 (lineal, polinomico,
# RBF) con los gamma de la grilla que el Ejercicio 10 recorrio de verdad, para
# que las dos listas sean comparables fila por fila.
candidatos = [("lineal", "", linear_kernel(Xtr))]
for q in (2, 3):
    candidatos.append(("polinomico", "grado %d" % q,
                       polynomial_kernel(Xtr, degree=q, gamma=1.0, coef0=1.0)))
for g in (0.0001, 0.001, 0.01, 0.1, 1.0):
    candidatos.append(("RBF", "gamma %g" % g, rbf_kernel(Xtr, gamma=g)))

filas = [(fam, par, alineamiento(K)) for fam, par, K in candidatos]
mejor = max(filas, key=lambda f: f[2])

with open(os.path.join(SAL, "alineamiento.csv"), "w", newline="",
          encoding="utf-8") as f:
```

```
    w = csv.writer(f)
    w.writerow(["familia", "parametro", "alineamiento"])
    for fam, par, a in filas:
        w.writerow([fam, par, "%.4f" % a])

# El RBF elegido por validacion cruzada en el Ejercicio 10, para poder decir en
# el texto si el atajo coincide o no con el resultado de entrenar.
gamma_cv = 0.01
a_cv = [a for fam, par, a in filas if par == "gamma %g" % gamma_cv][0]
meta = {
    "n_entrenamiento": n,
    "mejor_familia": mejor[0],
    "mejor_parametro": mejor[1],
    "mejor_alineamiento": round(mejor[2], 4),
    "gamma_elegido_por_cv": gamma_cv,
    "alineamiento_del_gamma_cv": round(a_cv, 4),
    "coincide_con_cv": mejor[1] == "gamma %g" % gamma_cv,
}
with open(os.path.join(SAL, "meta.json"), "w", encoding="utf-8") as f:
    json.dump(meta, f, indent=2, ensure_ascii=False)

print("alineamiento kernel-objetivo sobre %d casos de entrenamiento" % n)
for fam, par, a in filas:
    marca = "  <- mejor" if (fam, par) == (mejor[0], mejor[1]) else ""
    marca += "  <- elegido por CV en E10" if par == "gamma %g" % gamma_cv else ""
    print("  %-12s %-12s A = %.4f%s" % (fam, par, a, marca))
print("coincide con la validacion cruzada:", meta["coincide_con_cv"])
print("escrito en resultados/e5: alineamiento.csv, meta.json")
```

A.5 Atajos de regresión contra Kaplan-Meier y Cox en Rossi (Ejercicio 6)

```
"""Ejercicio 6: por qué la regresión común falla con censura, sobre los datos Rossi.

Datos Rossi (Rossi et al., 1980), los mismos de la clase 7: 432 liberados seguidos 52
semanas; `week` es la semana de re-arresto o de fin del seguimiento y `arrest` vale 1
si hubo re-arresto. Se comparan dos atajos de regresión con Kaplan-Meier y Cox.

Salida: codigo/resultados/e6/meta.json (el .Rmd lee todos los números de ahí).
"""

import json
from pathlib import Path

import numpy as np
from lifelines import CoxPHFitter, KaplanMeierFitter
from lifelines.datasets import load_rossi
from lifelines.statistics import logrank_test
from lifelines.utils import restricted_mean_survival_time
from sklearn.linear_model import LinearRegression

SEED = 42  # nada es aleatorio acá; se fija igual por regla del trabajo
np.random.seed(SEED)

OUT = Path(__file__).resolve().parents[1] / "resultados" / "e6"
OUT.mkdir(parents=True, exist_ok=True)
```

```
df = load_rossi()
T, E = df["week"], df["arrest"]
COVS = ["fin", "age", "race", "wexp", "mar", "paro", "prio"]
horizonte = int(T.max())

# Atajo 1: tirar los censurados y promediar solo los re-arrestados
atajo_descarta = float(T[E == 1].mean())
# Atajo 2: tratar el fin del seguimiento como si fuera el re-arresto
atajo_imputa = float(T.mean())
# Toda la censura de Rossi cae en la semana 52 (fin administrativo del estudio)
censura_unica = bool((T[E == 0] == horizonte).all())

km = KaplanMeierFitter().fit(T, E)
s_horizonte = float(km.survival_function_at_times(horizonte).iloc[0])
s_26 = float(km.survival_function_at_times(26).iloc[0])
mediana_km = km.median_survival_time_  # inf si S(t) nunca baja de 0,5
# Media restringida a 52 semanas (área bajo S(t)). Primero se la puso como "la media
# correcta" contra el atajo 2 y dio idéntica: con censura toda en t = 52, el atajo 2
# estima justamente esta media truncada, no el tiempo medio hasta el re-arresto.
rmst = float(restricted_mean_survival_time(km, t=horizonte))

# Efecto de la ayuda financiera (fin): regresión sobre re-arrestados contra Cox
ols = LinearRegression().fit(df.loc[E == 1, COVS], T[E == 1])
coef_ols_fin = float(ols.coef_[COVS.index("fin")])
# Probabilidad de re-arresto según fin, ignorando el tiempo (la "logística sin tiempo")
tasa_fin = df.groupby("fin")["arrest"].mean()

cph = CoxPHFitter().fit(df, duration_col="week", event_col="arrest")
res = cph.summary[["coef", "exp(coef)", "exp(coef) lower 95%", "exp(coef) upper 95%", "p"]]

lr = logrank_test(T[df.fin == 1], T[df.fin == 0], E[df.fin == 1], E[df.fin == 0])
km1 = KaplanMeierFitter().fit(T[df.fin == 1], E[df.fin == 1])
km0 = KaplanMeierFitter().fit(T[df.fin == 0], E[df.fin == 0])
meta = {
    "n": int(len(df)), "eventos": int(E.sum()), "censurados": int((E == 0).sum()),
    "pct_censura": float((E == 0).mean() * 100), "horizonte": horizonte,
    "atajo_descarta": atajo_descarta, "atajo_imputa": atajo_imputa, "rmst": rmst,
    "censura_unica": censura_unica,
    "s_horizonte": s_horizonte, "s_26": s_26,
    "mediana_km_finita": bool(np.isfinite(mediana_km)),
    "coef_ols_fin": coef_ols_fin,
    "tasa_arresto_fin1": float(tasa_fin[1]), "tasa_arresto_fin0": float(tasa_fin[0]),
    "hr_fin": float(res.loc["fin", "exp(coef)"]),
    "hr_fin_lo": float(res.loc["fin", "exp(coef) lower 95%"]),
    "hr_fin_hi": float(res.loc["fin", "exp(coef) upper 95%"]),
    "p_fin": float(res.loc["fin", "p"]),
    "hr_prio": float(res.loc["prio", "exp(coef)"]),
    "hr_age": float(res.loc["age", "exp(coef)"]),
    "c_index": float(cph.concordance_index_),
    "logrank_p": float(lr.p_value),
    "s_horizonte_fin1": float(km1.survival_function_at_times(horizonte).iloc[0]),
    "s_horizonte_fin0": float(km0.survival_function_at_times(horizonte).iloc[0]),
}
(OUT / "meta.json").write_text(json.dumps(meta, indent=2, ensure_ascii=False))
print(json.dumps(meta, indent=2, ensure_ascii=False))
```

A.6 t-SNE y clustering sobre dígitos (Ejercicio 7)

Produce `codigo/resultados/e7/`: las métricas, el barrido del radio de DBSCAN, los perfiles de los grupos, el ANOVA por componente y el mapa.

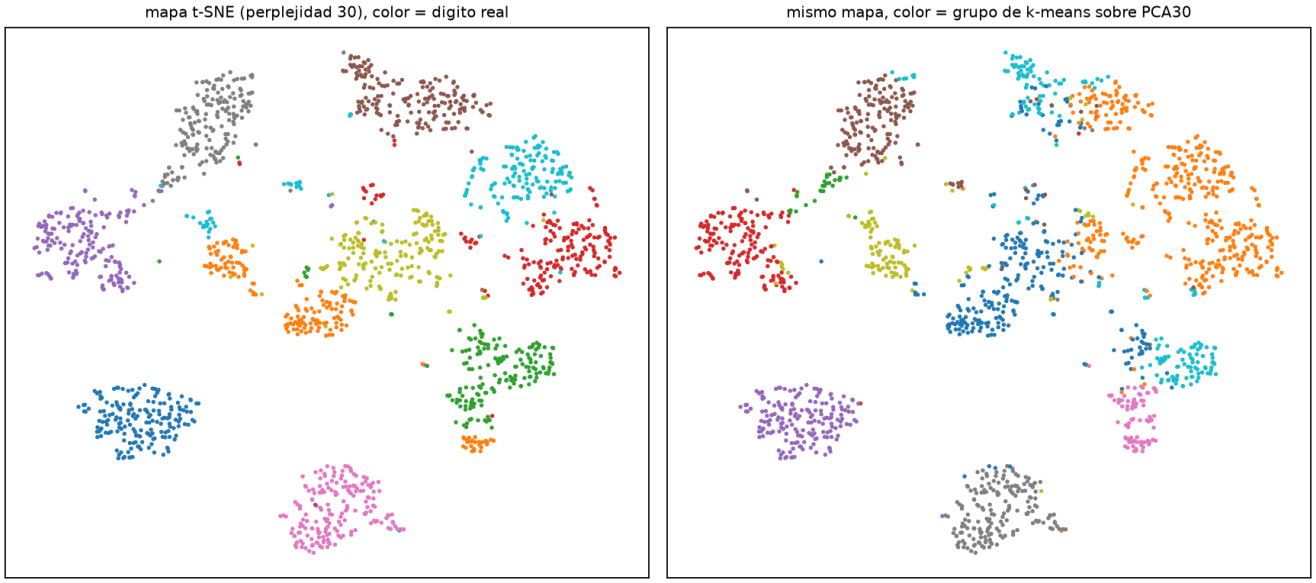


Figura 4: Proyección t-SNE de los 1797 dígitos. A la izquierda el color es el dígito real; a la derecha, el grupo que asignó k-means sobre las 30 componentes.

Nota. Elaboración propia. En el panel derecho se lee el índice de Rand de k-means: un grupo cubre dos nubes que a la izquierda son dígitos distintos y otras nubes aparecen partidas.

```
"""
Ejercicio 7 - t-SNE sobre digitos manuscritos

Aplica el pipeline de tres pasos que recomienda el apunte de la Clase 8:
  (1) explorar con t-SNE sobre datos estandarizados,
  (2) hacer el clustering sobre el espacio original (aca, sobre el PCA previo),
  (3) validar superponiendo las etiquetas sobre la proyeccion.

Nota personal sobre el paso 2: la tentacion es correr DBSCAN directamente sobre
las dos coordenadas del embedding, porque ahi los grupos se ven separados y
"funciona". El apunte advierte que eso esta mal, y el motivo es que las
distancias del mapa t-SNE no son confiables: el algoritmo preserva vecindad
local y distorsiona todo lo demas, asi que un eps ajustado sobre el embedding
no significa nada en el espacio de los datos.
"""

import os
import csv

AQUI = os.path.dirname(os.path.abspath(__file__))
SAL = os.path.join(AQUI, "..", "resultados", "e7")
os.makedirs(SAL, exist_ok=True)

import numpy as np
from sklearn.datasets import load_digits
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
```

```
from sklearn.manifold import TSNE
from sklearn.cluster import DBSCAN, KMeans
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import adjusted_rand_score, silhouette_score
from sklearn.neighbors import NearestNeighbors
from scipy.stats import f_oneway

RANDOM_STATE = 42

datos = load_digits()
X, y = datos.data, datos.target
M = {}
M["n"] = X.shape[0]; M["p"] = X.shape[1]; M["clases"] = len(np.unique(y))
print("dataset:", X.shape, "clases:", M["clases"])

X_std = StandardScaler().fit_transform(X)

# PCA previo a 30 componentes. El apunte lo recomienda para inicializar y
# acelerar; ademas filtra ruido antes de calcular las distancias que alimentan
# las probabilidades p_ij.
pca = PCA(n_components=30, random_state=RANDOM_STATE)
X30 = pca.fit_transform(X_std)
M["var_pca30"] = float(pca.explained_variance_ratio_.sum())
print("varianza explicada por 30 componentes: %.3f" % M["var_pca30"])

# --- Sensibilidad a la perplejidad ----------------------------------------
# Se prueban tres valores dentro del rango 5-50 que indica el apunte. La idea
# es verificar en carne propia la limitante: si la cantidad de grupos que se
# "ven" cambia con la perplejidad, el mapa no puede leerse como una particion.
embeddings = {}
for perp in [5, 30, 50]:
    emb = TSNE(n_components=2, perplexity=perp, init="pca",
               learning_rate="auto", random_state=RANDOM_STATE).fit_transform(X30)
    embeddings[perp] = emb
    sil = silhouette_score(emb, y)
    M["sil_p%d" % perp] = float(sil)
    print("perplexity=%2d -> silhouette de las clases reales en el mapa: %.3f"
          % (perp, sil))

# --- Clustering sobre el espacio original (no sobre el embedding) ----------
# Primer intento con DBSCAN. La primera grilla de eps arrancaba en 5 y en todo
# ese rango devolvia un unico cluster con ARI = 0; se lo leyo como prueba de que
# DBSCAN no separaba nada. Estaba mal ubicada: la distancia de cada punto a su
# 8.o vecino (el grafico de distancia k de Ester et al.) tiene la mediana cerca
# de 3,8, asi que con eps >= 5 casi todo queda conectado con todo. Se la extiende
# hacia abajo, con paso fino entre 2 y 4, que es donde DBSCAN encuentra grupos.
vecinos = NearestNeighbors(n_neighbors=8).fit(X30)
dist_k = vecinos.kneighbors(X30)[0][:, -1]  # incluye al propio punto, como min_samples
for q in [10, 50, 90]:
    M["kdist_q%d" % q] = float(np.percentile(dist_k, q))
print("distancia al 8.o vecino, cuantiles 10/50/90: %.2f %.2f %.2f"
      % (M["kdist_q10"], M["kdist_q50"], M["kdist_q90"]))

barrido = []
print("eps  clusters  ruido   ARI  ARI_sin_ruido")
for eps in [2.0, 2.25, 2.5, 2.75, 3.0, 3.25, 3.5, 3.75, 4.0, 4.5, 5, 7, 9, 12,
            16, 20]:
```

```
    lab = DBSCAN(eps=eps, min_samples=8).fit_predict(X30)
    n = len(set(lab)) - (1 if -1 in lab else 0)
    nr = lab != -1
    # ARI sobre todos los puntos (el ruido cuenta como un grupo mas) y aparte
    # sobre los que no son ruido: con eps chico el segundo sale alto solo porque
    # DBSCAN se queda con los nucleos faciles y tira el resto.
    a_nr = adjusted_rand_score(y[nr], lab[nr]) if nr.sum() > 0 else float("nan")
    barrido.append((eps, n, int((~nr).sum()), adjusted_rand_score(y, lab), a_nr))
    print("%4.2f %6d %8d  %.3f  %.3f" % barrido[-1])

# El eps que se informa es el de mayor ARI sobre todos los puntos. Se recalcula
# aparte para que el numero que cita el texto salga de un print y no de la tabla.
eps_mejor = max(barrido, key=lambda f: f[3])[0]
db = DBSCAN(eps=eps_mejor, min_samples=8).fit_predict(X30)
n_db = len(set(db)) - (1 if -1 in db else 0)
M["db_eps"] = eps_mejor
M["db_clusters"] = n_db
M["db_ruido"] = int((db == -1).sum())
M["ari_dbscan"] = float(adjusted_rand_score(y, db))
M["ari_dbscan_sin_ruido"] = float(adjusted_rand_score(y[db != -1], db[db != -1]))
print("DBSCAN sobre PCA30 (eps=%.2f): %d clusters, %d puntos de ruido, ARI=%.3f"
      % (eps_mejor, n_db, M["db_ruido"], M["ari_dbscan"]))

km = KMeans(n_clusters=10, n_init=10, random_state=RANDOM_STATE).fit_predict(X30)
M["ari_kmeans_pca"] = float(adjusted_rand_score(y, km))
print("KMeans k=10 sobre PCA30: ARI=%.3f" % M["ari_kmeans_pca"])

# --- Perfiles de los grupos (capitulo 5 de la guia, Assessing Clustering Results) ---
# El capitulo propone comparar primero las medias de las variables de entrada entre
# grupos y recien despues ir al test. Con 64 pixeles la tabla es ilegible, asi que se
# perfila sobre las primeras cinco componentes, que el PCA ya dejo ordenadas por
# varianza. La aclaracion que hace el propio capitulo y que conviene no perder: los
# p-valores de abajo NO son una prueba de hipotesis valida, porque los grupos se
# construyeron a partir de estos mismos datos. Sirven para ordenar cual componente
# separa mas, y para nada mas que eso.
print("\nperfiles de los 10 grupos de k-means, medias sobre PC1..PC5")
print("grupo     n" + "".join("%9s" % ("PC%d" % (j + 1)) for j in range(5)))
for c in range(10):
    m = km == c
    print("%5d %5d" % (c, m.sum())
          + "".join("%9.2f" % X30[m, j].mean() for j in range(5)))

print("\nANOVA de una via por componente sobre los grupos de k-means")
for j in range(5):
    F, p = f_oneway(*[X30[km == c, j] for c in range(10)])
    print("PC%d  F=%8.1f  p=%.3g" % (j + 1, F, p))

# --- Contraejemplo deliberado: clustering sobre el embedding ---------------
# Se deja en el codigo porque el contraste es el argumento: el ARI sobre el
# embedding sale MEJOR, y justamente por eso es una trampa. El mapa ya separo
# los grupos, asi que agrupar sobre el es medir el trabajo de t-SNE, no
# descubrir estructura en los datos.
km_emb = KMeans(n_clusters=10, n_init=10,
                random_state=RANDOM_STATE).fit_predict(embeddings[30])
M["ari_kmeans_emb"] = float(adjusted_rand_score(y, km_emb))
print("KMeans k=10 sobre el embedding t-SNE: ARI=%.3f (ver comentario)"
      % M["ari_kmeans_emb"])
```

```
# --- Paso 3 del pipeline: validacion visual ---------------------------------
# Superponer las etiquetas sobre la proyeccion es el paso 3 que el apunte recomienda y
# el unico que el informe describia sin ejecutar. Se grafican los dos paneles juntos a
# proposito: a la izquierda el digito verdadero y a la derecha el grupo de k-means, que
# es la unica forma honesta de mostrar donde el clustering acierta y donde parte un
# digito en dos.
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

emb = embeddings[30]
fig, ax = plt.subplots(1, 2, figsize=(11, 5))
ax[0].scatter(emb[:, 0], emb[:, 1], c=y, cmap="tab10", s=6, linewidths=0)
ax[0].set_title("mapa t-SNE (perplejidad 30), color = digito real", fontsize=9)
ax[1].scatter(emb[:, 0], emb[:, 1], c=km, cmap="tab10", s=6, linewidths=0)
ax[1].set_title("mismo mapa, color = grupo de k-means sobre PCA30", fontsize=9)
for a in ax:
    a.set_xticks([]); a.set_yticks([])
fig.tight_layout()
fig.savefig(os.path.join(SAL, "tsne_mapa.png"), dpi=160, bbox_inches="tight")
plt.close(fig)
print("\nescrito fig_ej07_tsne.png")


# --- Exportacion para el informe --------------------------------------------
# El .Rmd no ejecuta Python: lee estos CSV con read.csv(), asi que ningun
# numero del texto se tipea a mano.
M["ruido_eps_min"] = barrido[0][2]
M["ruido_eps_max"] = barrido[-1][2]

with open(os.path.join(SAL, "metricas.csv"), "w", newline="", encoding="utf-8") as f:
    w = csv.writer(f); w.writerow(["metrica", "valor"])
    for k, v in M.items():
        w.writerow([k, v])

with open(os.path.join(SAL, "barrido_eps.csv"), "w", newline="", encoding="utf-8") as f:
    w = csv.writer(f); w.writerow(["eps", "clusters", "ruido", "ari", "ari_sin_ruido"])
    for fila in barrido:
        w.writerow(["%.2f" % fila[0], fila[1], fila[2], "%.3f" % fila[3],
                    "%.3f" % fila[4]])

with open(os.path.join(SAL, "perfiles_kmeans.csv"), "w", newline="", encoding="utf-8") as f:
    w = csv.writer(f)
    w.writerow(["grupo", "n"] + ["PC%d" % (j + 1) for j in range(5)])
    for c in range(10):
        m = km == c
        w.writerow([c, int(m.sum())] + ["%.2f" % X30[m, j].mean() for j in range(5)])

with open(os.path.join(SAL, "anova_componentes.csv"), "w", newline="", encoding="utf-8") as f:
    w = csv.writer(f); w.writerow(["componente", "F"])
    for j in range(5):
        F, _ = f_oneway(*[X30[km == c, j] for c in range(10)])
        w.writerow(["PC%d" % (j + 1), "%.1f" % F])

# Paso 2 del capitulo, el que faltaba. El MANOVA va ANTES del ANOVA por
# variable, porque pregunta si los grupos difieren en el VECTOR completo y asi
```

```
# no infla el error de tipo I con una prueba por componente. Se calcula a mano
# con las matrices de dispersion para no sumar statsmodels: Lambda de Wilks es
# det(W)/det(T), con W la dispersion dentro de los grupos y T la total, y la
# aproximacion de Bartlett la lleva a una chi cuadrado. Lambda cerca de 0 es
# separacion total y cerca de 1 es ninguna. La advertencia del ANOVA vale
# igual: el p esta sesgado hacia la significacion porque los grupos salieron de
# estos mismos datos, asi que esto se lee como tamano de la diferencia.
P5 = X30[:, :5]
n_obs, p_var = P5.shape
k_gr = 10
W = np.zeros((p_var, p_var))
for c in range(k_gr):
    Z = P5[km == c] - P5[km == c].mean(axis=0)
    W += Z.T @ Z
Zt = P5 - P5.mean(axis=0)
T = Zt.T @ Zt
wilks = float(np.linalg.det(W) / np.linalg.det(T))
gl_manova = p_var * (k_gr - 1)
chi2 = float(-(n_obs - 1 - (p_var + k_gr) / 2) * np.log(wilks))

# CANDISC, el otro paso que el capitulo pide junto al ANOVA. Los ejes canonicos
# maximizan la separacion ENTRE grupos, que NO es lo mismo que la varianza
# total que ordena las componentes principales. Lo interesante es el contraste
# de los dos primeros ejes de cada cosa: si el canonico concentra mas
# separacion que lo que las dos primeras componentes explican de varianza, esa
# brecha es la razon de ser de CANDISC y lo que explica que el F de PC5 supere
# al de PC4.
cand = LinearDiscriminantAnalysis().fit(P5, km)
sep_can2 = float(cand.explained_variance_ratio_[:2].sum())
var_pca2 = float(pca.explained_variance_ratio_[:2].sum())

with open(os.path.join(SAL, "manova_candisc.csv"), "w", newline="",
          encoding="utf-8") as f:
    w = csv.writer(f); w.writerow(["metrica", "valor"])
    w.writerow(["wilks_lambda", "%.4f" % wilks])
    w.writerow(["chi2_bartlett", "%.0f" % chi2])
    w.writerow(["gl", gl_manova])
    w.writerow(["sep_canonica_2ejes", "%.3f" % sep_can2])
    w.writerow(["var_pca_2comp", "%.3f" % var_pca2])

print("\nMANOVA sobre PC1..PC5: Lambda de Wilks=%.4f, chi2 Bartlett=%.0f (gl=%d)"
      % (wilks, chi2, gl_manova))
print("CANDISC: los 2 primeros ejes canonicos concentran %.1f%% de la separacion; "
      "las 2 primeras componentes explican %.1f%% de la varianza total"
      % (100 * sep_can2, 100 * var_pca2))

# Paso 3 del perfilado, con un agregado propio. El capitulo cierra con el ANOVA
# por variable, cuyos p quedan sesgados porque los grupos se construyeron para
# maximizar la separacion. Un clasificador contra la etiqueta de grupo no tiene
# ese problema: no asume nada sobre la distribucion, y su importancia de
# variables dice lo mismo que buscaba el ANOVA sin pedir prestado un valor p.
# oob_score da ademas una medida de cuan separables son los grupos.
#
# Se entrena sobre las MISMAS 5 componentes de la tabla de perfiles, no sobre
# las 30, para que la importancia sea comparable con los F de arriba. Primero
# lo probe sobre las 30 y la importancia se reparte entre tantas componentes
# que no se puede contrastar con nada.
rf = RandomForestClassifier(n_estimators=500, oob_score=True, random_state=0,
```

```
                            n_jobs=-1)
rf.fit(X30[:, :5], km)
imp = rf.feature_importances_

with open(os.path.join(SAL, "importancia_rf.csv"), "w", newline="",
          encoding="utf-8") as f:
    w = csv.writer(f); w.writerow(["componente", "importancia"])
    for j in range(5):
        w.writerow(["PC%d" % (j + 1), "%.3f" % imp[j]])

M["rf_oob"] = "%.3f" % rf.oob_score_
with open(os.path.join(SAL, "metricas.csv"), "w", newline="",
          encoding="utf-8") as f:
    w = csv.writer(f); w.writerow(["metrica", "valor"])
    for k, v in M.items():
        w.writerow([k, v])

# Estabilidad entre semillas. Hasta aca todo salia de UNA corrida con semilla
# fija, y en el informe se afirma un ORDEN de importancia y una forma de la
# curva de siluetas. Afirmar un orden a partir de una corrida es leer ruido si
# las brechas son del tamano de la dispersion entre semillas, asi que hay que
# medirla en vez de suponerla. Se rehace el bosque con 8 semillas; el t-SNE no
# se rehace aca porque cuesta minutos y se verifico aparte (4 semillas, desvio
# de silueta 0,001).
ordenes, oobs = [], []
for sem in range(8):
    r = RandomForestClassifier(n_estimators=500, oob_score=True,
                               random_state=sem, n_jobs=-1).fit(X30[:, :5], km)
    oobs.append(r.oob_score_)
    ordenes.append(tuple(np.argsort(-r.feature_importances_)))
estable = len(set(ordenes)) == 1

with open(os.path.join(SAL, "estabilidad_rf.csv"), "w", newline="",
          encoding="utf-8") as f:
    w = csv.writer(f); w.writerow(["metrica", "valor"])
    w.writerow(["semillas", len(oobs)])
    w.writerow(["orden_identico", int(estable)])
    w.writerow(["oob_media", "%.4f" % (sum(oobs) / len(oobs))])
    w.writerow(["oob_min", "%.4f" % min(oobs)])
    w.writerow(["oob_max", "%.4f" % max(oobs)])

print("estabilidad: orden identico en %d/%d semillas, oob %.4f-%.4f"
      % (len(oobs) if estable else 0, len(oobs), min(oobs), max(oobs)))

print("escritos en resultados/e7: metricas.csv, barrido_eps.csv, "
      "perfiles_kmeans.csv, anova_componentes.csv, manova_candisc.csv, "
      "importancia_rf.csv, estabilidad_rf.csv, "
      "tsne_mapa.png")
```

A.7 Pipeline Opción 1 (Ejercicio 10)

```
"""
Ejercicio 10, Opción 1: pipeline comparativo de SVM, Random Forest, Gradient
Boosting y MLP sobre una tarea de clasificación y una de regresión.

Esquema de validación:
  1. Hold-out externo del 20 % (estratificado en clasificación), que no se toca
     durante la búsqueda de hiperparámetros.
```

```
  2. Sobre el 80 % de entrenamiento, RandomizedSearchCV con k-fold de 5 bloques.
  3. La configuración elegida se reentrena sobre todo el 80 % y se evalúa una
     sola vez sobre el hold-out.
El preproceso (StandardScaler) vive dentro del Pipeline. Semilla 42.

Correr desde la raíz del repositorio:
  python codigo/e10_pipeline/pipeline_opcion1.py

Salidas en codigo/resultados/e10/: comparacion_modelos.csv, espacio_busqueda.csv,
hiperparametros_elegidos.csv, comparacion_modelos.png y meta.json.
"""

from __future__ import annotations

import json
import platform
import time
import warnings
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt  # noqa: E402
import numpy as np  # noqa: E402
import pandas as pd  # noqa: E402
import sklearn  # noqa: E402
from matplotlib.ticker import FuncFormatter, MultipleLocator  # noqa: E402
from sklearn.datasets import fetch_california_housing, load_breast_cancer  # noqa: E402
from sklearn.ensemble import (  # noqa: E402
    GradientBoostingClassifier,
    GradientBoostingRegressor,
    RandomForestClassifier,
    RandomForestRegressor,
)
from sklearn.metrics import (  # noqa: E402
    accuracy_score,
    confusion_matrix,
    f1_score,
    r2_score,
    roc_auc_score,
    root_mean_squared_error,
)
from sklearn.model_selection import (  # noqa: E402
    KFold,
    RandomizedSearchCV,
    StratifiedKFold,
    cross_val_score,
    train_test_split,
)
from sklearn.neural_network import MLPClassifier, MLPRegressor  # noqa: E402
from sklearn.pipeline import Pipeline  # noqa: E402
from sklearn.preprocessing import StandardScaler  # noqa: E402
from sklearn.svm import SVC, SVR  # noqa: E402

SEED = 42
TEST_SIZE = 0.20
CV_SPLITS = 5
```

```
# 30 combinaciones por modelo: con grillas de 24 a 150 puntos alcanza para que
# cada valor de cada hiperparámetro aparezca varias veces (Bergstra y Bengio,
# 2012). Cuando la grilla tiene 30 puntos o menos, la búsqueda aleatoria la
# recorre completa y equivale a un GridSearchCV; eso se registra en el CSV.
N_ITER = 30
N_JOBS = -1  # 8 núcleos; cada fit es independiente y lleva su propia semilla.
# Tiempos observados en la corrida que generó los resultados del informe (Apple
# Silicon, 8 núcleos): clasificación SVM 2 s, RF 7 s, GB 6 s, MLP 5 s; regresión
# SVM 39 s, RF 66 s, GB 172 s, MLP 31 s; total 331 s. Una corrida previa, con la
# máquina ocupada por otros procesos, tardó 637 s (GB de regresión 313 s): el
# tiempo depende de la carga.
# Los valores de cada corrida quedan en meta.json.
# California Housing tiene 20 640 filas. En una prueba exploratoria aparte, un
# SVR con C = 1000 y gamma = 1 tardó 46,5 s por fit sobre un bloque de
# entrenamiento de una submuestra de 8000 filas (5120 filas) y 80,7 s con una de
# 12 000. Se submuestrea a 8000 antes de la partición para que el SVR entre en
# el presupuesto de tiempo y los cuatro modelos compitan sobre los mismos datos.
# Esa prueba no se repite en cada corrida (cuesta unos 2 min); sus tiempos se
# copian a meta.json para que el informe los lea de ahí.
N_SUB_REG = 8000
TIEMPO_SVR_PRUEBA = {
    "C": 1000, "gamma": 1.0, "filas_submuestra": [8000, 12000],
    "segundos_por_fit": [46.5, 80.7],
}

OUT = Path(__file__).resolve().parents[1] / "resultados" / "e10"

ETIQUETAS = {
    "SVM": "SVM",
    "RF": "Random Forest",
    "GB": "Gradient Boosting",
    "MLP": "MLP",
}


def _cv(tarea: str):
    if tarea == "clasificacion":
        return StratifiedKFold(n_splits=CV_SPLITS, shuffle=True, random_state=SEED)
    return KFold(n_splits=CV_SPLITS, shuffle=True, random_state=SEED)


def specs_clasificacion(p: int) -> list[dict]:
    """Modelos y grillas para Breast Cancer (p = 30 predictores)."""
    return [
        {
            "modelo": "SVM",
            # SVC sin probability=True no usa random_state: no se fija.
            "pipe": Pipeline([("scaler", StandardScaler()), ("model", SVC())]),
            # Grilla logarítmica en C y gamma, como sugieren Hsu et al. (2003).
            # En la corrida anterior el óptimo fue C = 10 y gamma = 0,01 sobre
            # una grilla de C hasta 100: se agregan 0,01 y 1000 en C y 0,1 y 1
            # en gamma para que ese óptimo quede en el interior.
            "grilla": {
                "model__C": [0.01, 0.1, 1.0, 10.0, 100.0, 1000.0],
                "model__gamma": [1e-4, 1e-3, 1e-2, 1e-1, 1.0],
            },
            "fijos": {"model__kernel": "rbf"},
            "limites": {},
```

```
    },
    {
        "modelo": "RF",
        "pipe": Pipeline(
            [
                (
                    "model",
                    RandomForestClassifier(
                        n_estimators=300, random_state=SEED, n_jobs=1
                    ),
                )
            ]
        ),
        # Más árboles no sobreajustan, solo cuestan tiempo (Breiman, 2001):
        # se fija n_estimators = 300 y se busca sobre m (max_features) y el
        # tamaño mínimo de hoja. El max_depth = 20 de la corrida anterior
        # quedó en el borde; se reemplaza por min_samples_leaf, que controla
        # la misma complejidad sin un tope arbitrario de profundidad.
        # m = 30 es bagging puro; m = 2 casi no deja elegir variable.
        "grilla": {
            "model__max_features": [2, 4, 6, 10, 15, p],
            "model__min_samples_leaf": [1, 2, 4, 8],
        },
        "fijos": {"model__n_estimators": 300},
        # Bordes que no se pueden extender: m = p y hoja de 1 observación.
        "limites": {
            "model__max_features": [p],
            "model__min_samples_leaf": [1],
        },
    },
    {
        "modelo": "GB",
        "pipe": Pipeline(
            [("model", GradientBoostingClassifier(random_state=SEED))]
        ),
        # Antes ganó lr 0,1 / profundidad 4 / 50 etapas, con 0,1 y 4 como
        # máximos de la grilla. Se amplía la profundidad a 6 y se agregan 25
        # y 400 etapas; profundidad 1 son stumps (Ejercicio 8). Con lr hasta
        # 0,3 una corrida previa eligió 0,3, otra vez en el borde: se agregó
        # 1,0, el tope natural de nu (paso completo, sin contracción). En la
        # corrida final ganó de nuevo 0,3, ahora en el interior.
        "grilla": {
            "model__n_estimators": [25, 50, 100, 200, 400],
            "model__learning_rate": [0.01, 0.03, 0.1, 0.3, 1.0],
            "model__max_depth": [1, 2, 3, 4, 6],
        },
        "fijos": {},
        "limites": {"model__max_depth": [1], "model__learning_rate": [1.0]},
    },
    {
        "modelo": "MLP",
        # Prueba de convergencia (función prueba_parada_temprana, 5 folds,
        # una capa de 64, sin búsqueda): con early_stopping el F1 medio cae
        # fuerte con lr 0,0003; sin early_stopping sube y no hay avisos de
        # no convergencia con max_iter = 1000. Con 364 filas de entrenamiento en
        # cada vuelta, el 10 % interno de validación son unas 36 filas y corta pronto.
        # Se apaga early_stopping y se sube max_iter a 2000 por margen. La
        # configuración elegida terminó en 407 épocas, lejos del tope.
```

```
            "pipe": Pipeline(
                [
                    ("scaler", StandardScaler()),
                    (
                        "model",
                        MLPClassifier(
                            max_iter=2000, early_stopping=False, random_state=SEED
                        ),
                    ),
                ]
            ),
            # El lr 0,01 elegido antes era el máximo; ahora llega a 0,03. Una
            # corrida previa eligió 0,0003, el mínimo de entonces: se agregó
            # 0,0001 y en la corrida final 0,0003 ya queda en el interior.
            # alpha recorre seis órdenes de magnitud, de casi nula a fuerte.
            "grilla": {
                "model__hidden_layer_sizes": [(16,), (32,), (64,), (128,), (64, 32)],
                "model__alpha": [1e-5, 1e-4, 1e-3, 1e-2, 1e-1, 1.0],
                "model__learning_rate_init": [1e-4, 3e-4, 1e-3, 3e-3, 1e-2, 3e-2],
            },
            "fijos": {"model__max_iter": 2000, "model__early_stopping": False},
            "limites": {},
        },
    ]


def specs_regresion(p: int) -> list[dict]:
    """Modelos y grillas para California Housing (p = 8 predictores)."""
    return [
        {
            "modelo": "SVM",
            "pipe": Pipeline([("scaler", StandardScaler()), ("model", SVR())]),
            # Antes ganó C = 10 y epsilon = 0,2, ambos en el máximo. Se sube C a
            # 100 y no a 1000: C = 1000 con gamma = 1 tardó 46 s por fit y la
            # búsqueda completa no entraba en el presupuesto. epsilon está en
            # unidades de la respuesta (100 000 USD): 0,8 es un tubo muy ancho.
            # gamma = 1/8 (la regla "scale" con datos estandarizados) cae entre
            # 0,1 y 0,3 de esta grilla.
            "grilla": {
                "model__C": [0.1, 1.0, 10.0, 100.0],
                "model__gamma": [0.01, 0.03, 0.1, 0.3, 1.0],
                "model__epsilon": [0.05, 0.1, 0.2, 0.4, 0.8],
            },
            "fijos": {"model__kernel": "rbf"},
            "limites": {},
        },
        {
            "modelo": "RF",
            "pipe": Pipeline(
                [
                    (
                        "model",
                        RandomForestRegressor(
                            n_estimators=300, random_state=SEED, n_jobs=1
                        ),
                    )
                ]
            ),
```

```
        # Con p = 8, m recorre todos los valores útiles de 1 a 8 (m = 8 es
        # bagging). La hoja llega a 16 para dar margen de suavizado.
        "grilla": {
            "model__max_features": [1, 2, 3, 4, 6, p],
            "model__min_samples_leaf": [1, 2, 4, 8, 16],
        },
        "fijos": {"model__n_estimators": 300},
        "limites": {
            "model__max_features": [1, p],
            "model__min_samples_leaf": [1],
        },
    },
    {
        "modelo": "GB",
        "pipe": Pipeline(
            [("model", GradientBoostingRegressor(random_state=SEED))]
        ),
        # Antes ganó lr 0,1 / profundidad 3 / 200 etapas, los tres en el
        # máximo. Se agregan lr 0,3, profundidades 4 a 8 y más etapas.
        # Con tope en 800 etapas ganó 800 (borde): se agregó 1600 y ganó 1600
        # con lr 0,03, otra vez en el borde. Es el canje entre nu y la
        # cantidad de etapas (Ejercicio 8). No se agregó 3200: la búsqueda de
        # GB ya es la más cara de la corrida (de 3 a 5 min según la carga) y
        # con nu chico más etapas suelen mejorar poco. Se informa como
        # limitación.
        "grilla": {
            "model__n_estimators": [100, 200, 400, 800, 1600],
            "model__learning_rate": [0.01, 0.03, 0.1, 0.3],
            "model__max_depth": [2, 3, 4, 6, 8],
        },
        "fijos": {},
        "limites": {},
    },
    {
        "modelo": "MLP",
        # Con 5120 filas por fold el 10 % de validación interna son 512
        # filas: ahí early_stopping sí es estable y acota el tiempo. La
        # configuración elegida se detuvo en la época 62.
        "pipe": Pipeline(
            [
                ("scaler", StandardScaler()),
                (
                    "model",
                    MLPRegressor(
                        max_iter=2000,
                        early_stopping=True,
                        n_iter_no_change=20,
                        random_state=SEED,
                    ),
                ),
            ]
        ),
        "grilla": {
            "model__hidden_layer_sizes": [
                (32,), (64,), (128,), (64, 32), (128, 64)
            ],
            "model__alpha": [1e-5, 1e-4, 1e-3, 1e-2, 1e-1, 1.0],
            "model__learning_rate_init": [3e-4, 1e-3, 3e-3, 1e-2, 3e-2],
```

```
            },
            "fijos": {
                "model__max_iter": 2000,
                "model__early_stopping": True,
                "model__n_iter_no_change": 20,
            },
            "limites": {},
        },
    ]


def _fmt(v) -> str:
    """Valor legible para las tablas del informe: coma decimal y punto de miles,
    la misma convención que usa el texto del .Rmd."""
    if isinstance(v, tuple):
        return "(" + ", ".join(str(x) for x in v) + ")"
    if isinstance(v, bool):
        return "sí" if v else "no"
    if isinstance(v, float) and not (v.is_integer() and v >= 1000):
        return np.format_float_positional(v, trim="-").replace(".", ",")
    if isinstance(v, (int, float)) and v >= 1000:
        return f"{int(v):,}".replace(",", ".")
    return str(v)


def _bordes(spec: dict, best: dict, tarea: str) -> list[dict]:
    """Marca qué hiperparámetros elegidos quedaron en un extremo de su grilla."""
    filas = []
    for hp, valores in spec["grilla"].items():
        elegido = best[hp]
        ordinal = all(isinstance(v, (int, float)) for v in valores)
        if ordinal:
            orden = sorted(valores)
            en_borde = elegido in (orden[0], orden[-1])
            extensible = en_borde and elegido not in spec["limites"].get(hp, [])
        else:
            # hidden_layer_sizes no tiene un orden natural único: no se evalúa.
            en_borde, extensible = None, None
        filas.append(
            {
                "tarea": tarea,
                "modelo": ETIQUETAS[spec["modelo"]],
                "hiperparametro": hp.replace("model__", ""),
                "valor": _fmt(elegido),
                "en_borde": en_borde,
                "borde_extensible": extensible,
            }
        )
    return filas


def correr_tarea(tarea, X, y, specs, scoring, meta):
    strat = y if tarea == "clasificacion" else None
    Xtr, Xte, ytr, yte = train_test_split(
        X, y, test_size=TEST_SIZE, stratify=strat, random_state=SEED
    )
    meta["particion"][tarea] = {"train": int(len(ytr)), "test": int(len(yte))}
    if tarea == "clasificacion":
```

```
        meta["particion"][tarea]["malignos_test"] = int(yte.sum())

    filas, grillas, elegidos, mejores = [], [], [], {}
    for spec in specs:
        n_grilla = int(np.prod([len(v) for v in spec["grilla"].values()]))
        n_eval = min(N_ITER, n_grilla)
        search = RandomizedSearchCV(
            spec["pipe"],
            spec["grilla"],
            n_iter=n_eval,
            scoring=scoring,
            refit=list(scoring)[0],
            cv=_cv(tarea),
            random_state=SEED,
            n_jobs=N_JOBS,
        )
        t0 = time.perf_counter()
        search.fit(Xtr, ytr)
        seg = time.perf_counter() - t0
        best = search.best_estimator_
        res, i = search.cv_results_, search.best_index_
        clave = list(scoring)[0]
        signo = -1.0 if tarea == "regresion" else 1.0
        fila = {
            "tarea": tarea,
            "modelo": ETIQUETAS[spec["modelo"]],
            "cv_media": signo * float(res[f"mean_test_{clave}"][i]),
            "cv_sd": float(res[f"std_test_{clave}"][i]),
            "combinaciones_grilla": n_grilla,
            "combinaciones_evaluadas": n_eval,
            "cubre_grilla": n_eval == n_grilla,
            "segundos": round(seg, 1),
            "best_params": json.dumps(
                {k.replace("model__", ""): v for k, v in search.best_params_.items()},
                sort_keys=True,
            ),
        }
        if tarea == "clasificacion":
            pred = best.predict(Xte)
            # roc_auc acepta decision_function (SVC) o predict_proba (resto).
            if hasattr(best, "predict_proba"):
                score = best.predict_proba(Xte)[:, 1]
            else:
                score = best.decision_function(Xte)
            tn, fp, fn, tp = confusion_matrix(yte, pred).ravel()
            fila.update(
                {
                    "test": f1_score(yte, pred),
                    "cv_accuracy": float(res["mean_test_accuracy"][i]),
                    "test_accuracy": accuracy_score(yte, pred),
                    "test_auc": roc_auc_score(yte, score),
                    "test_fn": int(fn),
                    "test_fp": int(fp),
                }
            )
        else:
            pred = best.predict(Xte)
            fila.update(
```

```
                {
                    "test": root_mean_squared_error(yte, pred),
                    "cv_r2": float(res["mean_test_r2"][i]),
                    "test_r2": r2_score(yte, pred),
                }
            )
        filas.append(fila)
        mejores[spec["modelo"]] = search.best_params_
        elegidos += _bordes(spec, search.best_params_, tarea)
        for hp, valores in {**spec["grilla"], **spec["fijos"]}.items():
            fijo = hp in spec["fijos"]
            grillas.append(
                {
                    "tarea": tarea,
                    "modelo": ETIQUETAS[spec["modelo"]],
                    "hiperparametro": hp.replace("model__", ""),
                    "valores": _fmt(valores)
                    if fijo
                    else ", ".join(_fmt(v) for v in valores),
                    "fijo": fijo,
                }
            )
        if spec["modelo"] == "MLP":
            meta["mlp_epocas_mejor"][tarea] = int(best.named_steps["model"].n_iter_)
        print(
            f"[{tarea}] {spec['modelo']}: CV {fila['cv_media']:.4f} "
            f"± {fila['cv_sd']:.4f} | test {fila['test']:.4f} | {seg:.0f} s"
        )
    return filas, grillas, elegidos, mejores, (Xtr, ytr)


def prueba_escalador_arboles(Xtr, ytr, mejores, meta) -> None:
    """RF y GB con y sin StandardScaler, con la configuración elegida.

    Los árboles cortan por umbrales sobre una variable por vez; un cambio de
    escala monótono mueve los umbrales pero no el orden de las observaciones.
    Se espera el mismo F1. Corrida registrada (clasificación, 5 folds): RF
    0,9552 con y sin escalador; GB 0,9703 con y sin escalador. Los números son
    idénticos: el escalador solo agrega un paso, por eso queda fuera de los
    Pipeline de árboles.
    """
    base = {
        "RF": RandomForestClassifier(n_estimators=300, random_state=SEED, n_jobs=1),
        "GB": GradientBoostingClassifier(random_state=SEED),
    }
    salida = {}
    for nombre, est in base.items():
        params = {k.replace("model__", ""): v for k, v in mejores[nombre].items()}
        est.set_params(**params)
        res = {}
        for con in (False, True):
            pasos = [("scaler", StandardScaler())] if con else []
            pipe = Pipeline(pasos + [("model", est)])
            s = cross_val_score(
                pipe, Xtr, ytr, cv=_cv("clasificacion"), scoring="f1", n_jobs=N_JOBS
            )
            res["con_escalador" if con else "sin_escalador"] = round(float(s.mean()), 4)
        salida[ETIQUETAS[nombre]] = res
```

```
        print(f"[escalador] {nombre}: {res}")
    meta["prueba_escalador_arboles_f1_cv"] = salida


def prueba_parada_temprana(Xtr, ytr, meta) -> None:
    """MLP (64,) con y sin early_stopping, dos tasas, CV de 5 bloques.

    Corrida registrada: con parada, F1 0,8628 (lr 0,0003) y 0,9620 (lr 0,03);
    sin parada, 0,9644 y 0,9646; 0 avisos de no convergencia. Se corre con
    n_jobs = 1 para poder contar los ConvergenceWarning en este proceso.
    """
    tasas = [3e-4, 3e-2]
    # validation_fraction por defecto (0,1): la fracción interna que usa la parada.
    salida = {"tasas": tasas, "fraccion_validacion": MLPClassifier().validation_fraction}
    avisos = 0
    for es in (True, False):
        f1s = []
        for lr in tasas:
            mlp = MLPClassifier(
                (64,), learning_rate_init=lr, max_iter=1000, early_stopping=es,
                n_iter_no_change=20, random_state=SEED,
            )
            pipe = Pipeline([("scaler", StandardScaler()), ("model", mlp)])
            with warnings.catch_warnings(record=True) as w:
                warnings.simplefilter("always")
                s = cross_val_score(pipe, Xtr, ytr, cv=_cv("clasificacion"), scoring="f1")
            avisos += sum("converge" in str(x.message).lower() for x in w)
            f1s.append(round(float(s.mean()), 4))
        salida["con_parada" if es else "sin_parada"] = f1s
    salida["avisos_no_convergencia"] = avisos
    meta["prueba_parada_temprana_mlp_f1_cv"] = salida
    print(f"[parada temprana] {salida}")


def figura(df: pd.DataFrame, outfile: Path) -> None:
    # Con 2 decimales fijos, ticks cada 0,025 salían redondeados (0,575 -> "0,57").
    # Se fija el paso a 0,01 / 0,02 para que cada rótulo sea exacto.
    coma = FuncFormatter(lambda v, _: f"{v:.2f}".replace(".", ","))
    paso = {"clasificacion": 0.01, "regresion": 0.02}
    paneles = [
        ("clasificacion", "F1 de la clase maligna", "Clasificación: Breast Cancer"),
        ("regresion", "RMSE (100 000 USD)", "Regresión: California Housing"),
    ]
    # Primera versión a 9 x 3,6 pulgadas: en el PDF, reducida al ancho de texto,
    # las etiquetas quedaban ilegibles. Con un lienzo menor y las mismas fuentes,
    # el texto queda más grande en proporción a la figura.
    fig, axes = plt.subplots(1, 2, figsize=(7, 2.9))
    for ax, (tarea, ylab, titulo) in zip(axes, paneles):
        sub = df[df["tarea"] == tarea].reset_index(drop=True)
        x = np.arange(len(sub))
        ax.errorbar(
            x - 0.1, sub["cv_media"], yerr=sub["cv_sd"], fmt="o", color="#1f4e79",
            capsize=4, label="CV interna (media ± DE)",
        )
        ax.plot(x + 0.1, sub["test"], "D", color="#c0504d", label="Hold-out (test)")
        ax.set_xticks(x, [m.replace(" ", "\n") for m in sub["modelo"]], fontsize=9)
        ax.tick_params(axis="y", labelsize=9)
        ax.set_xlim(-0.5, len(sub) - 0.5)
```

```
        ax.set_ylabel(ylab, fontsize=9)
        ax.set_title(titulo, fontsize=10)
        ax.yaxis.set_major_locator(MultipleLocator(paso[tarea]))
        ax.yaxis.set_major_formatter(coma)
        ax.grid(axis="y", alpha=0.3)
    h, lab = axes[0].get_legend_handles_labels()
    fig.legend(h, lab, loc="lower center", ncol=2, fontsize=9, frameon=False)
    fig.tight_layout(rect=(0, 0.08, 1, 1))
    fig.savefig(outfile, dpi=200)
    plt.close(fig)


def main() -> None:
    t_total = time.perf_counter()
    OUT.mkdir(parents=True, exist_ok=True)
    meta = {
        "seed": SEED,
        "sklearn": sklearn.__version__,
        "python": platform.python_version(),
        "test_size": TEST_SIZE,
        "cv_splits": CV_SPLITS,
        "n_iter": N_ITER,
        "particion": {},
        "mlp_epocas_mejor": {},
    }

    # Clasificación. En load_breast_cancer el 0 es maligno y el 1 benigno; se
    # recodifica para que la clase positiva (1) sea maligno y el F1 se calcule
    # sobre la clase que interesa detectar.
    bc = load_breast_cancer()
    Xc, yc = bc.data, (bc.target == 0).astype(int)
    meta["breast_cancer"] = {
        "n": int(len(yc)), "p": int(Xc.shape[1]), "malignos": int(yc.sum())
    }
    sc_clf = {"f1": "f1", "accuracy": "accuracy"}
    f_c, g_c, e_c, mej_c, (Xtr_c, ytr_c) = correr_tarea(
        "clasificacion", Xc, yc, specs_clasificacion(Xc.shape[1]), sc_clf, meta
    )
    prueba_escalador_arboles(Xtr_c, ytr_c, mej_c, meta)
    prueba_parada_temprana(Xtr_c, ytr_c, meta)
    meta["tiempo_svr_prueba_previa_s"] = TIEMPO_SVR_PRUEBA

    # Regresión. La respuesta es el valor mediano de la vivienda por grupo
    # censal, en unidades de 100 000 USD (tope en 5,00001 en los datos).
    Xr, yr = fetch_california_housing(return_X_y=True)
    meta["california_housing"] = {"n": int(len(yr)), "p": int(Xr.shape[1])}
    rng = np.random.default_rng(SEED)
    idx = rng.choice(len(yr), size=N_SUB_REG, replace=False)
    Xr, yr = Xr[idx], yr[idx]
    meta["california_housing"]["n_submuestra"] = N_SUB_REG
    sc_reg = {"rmse": "neg_root_mean_squared_error", "r2": "r2"}
    f_r, g_r, e_r, _, _ = correr_tarea(
        "regresion", Xr, yr, specs_regresion(Xr.shape[1]), sc_reg, meta
    )

    comp = pd.DataFrame(f_c + f_r)
    comp.to_csv(OUT / "comparacion_modelos.csv", index=False)
    pd.DataFrame(g_c + g_r).to_csv(OUT / "espacio_busqueda.csv", index=False)
```

```
    pd.DataFrame(e_c + e_r).to_csv(OUT / "hiperparametros_elegidos.csv", index=False)
    figura(comp, OUT / "comparacion_modelos.png")

    meta["segundos_por_busqueda"] = {
        f"{r['tarea']}/{r['modelo']}": r["segundos"] for r in f_c + f_r
    }
    meta["segundos_total"] = round(time.perf_counter() - t_total, 1)
    (OUT / "meta.json").write_text(json.dumps(meta, indent=2, ensure_ascii=False))
    print(f"Total: {meta['segundos_total']} s. Salidas en {OUT}")


if __name__ == "__main__":
    main()
```

Referencias

Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. *3rd International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/1409.0473

Belkin, M., Hsu, D., Ma, S., & Mandal, S. (2019). Reconciling modern machine-learning practice and the classical bias-variance trade-off. *Proceedings of the National Academy of Sciences, 116*(32), 15849-15854. https://doi.org/10.1073/pnas.1903070116

Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research, 13*, 281-305. http://jmlr.org/papers/v13/bergstra12a.html

Breiman, L. (1996). Bagging predictors. *Machine Learning, 24*(2), 123-140. https://doi.org/10.1007/BF00058655

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5-32. https://doi.org/10.1023/A:1010933404324

Burges, C. J. C. (1998). A tutorial on support vector machines for pattern recognition. *Data Mining and Knowledge Discovery, 2*(2), 121-167. https://doi.org/10.1023/A:1009715923555

Campello, R. J. G. B., Moulavi, D., & Sander, J. (2013). Density-based clustering based on hierarchical density estimates. *Advances in Knowledge Discovery and Data Mining (PAKDD 2013)*, 160-172. https://doi.org/10.1007/978-3-642-37456-2_14

Cawley, G. C., & Talbot, N. L. C. (2010). On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research, 11*, 2079-2107. http://jmlr.org/papers/v11/cawley10a.html

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-794. https://doi.org/10.1145/2939672.2939785

Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning, 20*(3), 273-297. https://doi.org/10.1007/BF00994018

Cox, D. R. (1972). Regression models and life-tables. *Journal of the Royal Statistical Society: Series B (Methodological), 34*(2), 187-202. https://doi.org/10.1111/j.2517-6161.1972.tb00899.x

Cristianini, N., Shawe-Taylor, J., Elisseeff, A., & Kandola, J. (2002). On kernel-target alignment. *Advances in Neural Information Processing Systems, 14*, 367-373.

Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems, 2*(4), 303-314. https://doi.org/10.1007/BF02551274

Davidson-Pilon, C. (2019). lifelines: Survival analysis in Python. *Journal of Open Source Software, 4*(40), 1317. https://doi.org/10.21105/joss.01317

Davis, L. (1985). Job shop scheduling with genetic algorithms. En J. J. Grefenstette (Ed.), *Proceedings of the First International Conference on Genetic Algorithms and their Applications* (pp. 136-140). Lawrence Erlbaum Associates.

Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96)*, 226-231.

Feng, G. (2024). Feature selection algorithm based on optimized genetic algorithm and the application in high-dimensional data processing. *PLOS ONE, 19*(5), e0303088. https://doi.org/10.1371/journal.pone.0303088

Fortin, F.-A., De Rainville, F.-M., Gardner, M.-A., Parizeau, M., & Gagné, C. (2012). DEAP: Evolutionary algorithms made easy. *Journal of Machine Learning Research, 13*, 2171-2175. http://jmlr.org/papers/v13/fortin12a.html

Freund, Y., & Schapire, R. E. (1997). A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences, 55*(1), 119-139. https://doi.org/10.1006/jcss.1997.1504

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics, 29*(5), 1189-1232. https://doi.org/10.1214/aos/1013203451

Gers, F. A., Schmidhuber, J., & Cummins, F. (2000). Learning to forget: Continual prediction with LSTM. *Neural Computation, 12*(10), 2451-2471. https://doi.org/10.1162/089976600300015015

Geurts, P., Ernst, D., & Wehenkel, L. (2006). Extremely randomized trees. *Machine Learning, 63*(1), 3-42. https://doi.org/10.1007/s10994-006-6226-1

Goldberg, D. E. (1989). *Genetic algorithms in search, optimization, and machine learning*. Addison-Wesley.

Gönen, M., & Alpaydın, E. (2011). Multiple kernel learning algorithms. *Journal of Machine Learning Research, 12*, 2211-2268. http://jmlr.org/papers/v12/gonen11a.html

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.

Grambsch, P. M., & Therneau, T. M. (1994). Proportional hazards tests and diagnostics based on weighted residuals. *Biometrika, 81*(3), 515-526. https://doi.org/10.1093/biomet/81.3.515

Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? *Advances in Neural Information Processing Systems, 35*, 507-520. https://doi.org/10.52202/068431-0037

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770-778. https://doi.org/10.1109/CVPR.2016.90

Hinton, G. E., & Salakhutdinov, R. R. (2006). Reducing the dimensionality of data with neural networks. *Science, 313*(5786), 504-507. https://doi.org/10.1126/science.1127647

Ho, T. K. (1995). Random decision forests. *Proceedings of the Third International Conference on Document Analysis and Recognition (ICDAR)*, 278-282. https://doi.org/10.1109/ICDAR.1995.598994

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation, 9*(8), 1735-1780. https://doi.org/10.1162/neco.1997.9.8.1735

Holland, J. H. (1975). *Adaptation in natural and artificial systems*. University of Michigan Press.

Hsu, C.-W., Chang, C.-C., & Lin, C.-J. (2003). *A practical guide to support vector classification* [Informe técnico]. Department of Computer Science, National Taiwan University. https://www.csie.ntu.edu.tw/~cjlin/papers/guide/guide.pdf

Hsu, C.-W., & Lin, C.-J. (2002). A comparison of methods for multiclass support vector machines. *IEEE Transactions on Neural Networks, 13*(2), 415-425. https://doi.org/10.1109/72.991427

Ishwaran, H., Kogalur, U. B., Blackstone, E. H., & Lauer, M. S. (2008). Random survival forests. *The Annals of Applied Statistics, 2*(3), 841-860. https://doi.org/10.1214/08-AOAS169

Janitza, S., & Hornung, R. (2018). On the overestimation of random forest's out-of-bag error. *PLOS ONE, 13*(8), e0201904. https://doi.org/10.1371/journal.pone.0201904

Kaplan, E. L., & Meier, P. (1958). Nonparametric estimation from incomplete observations. *Journal of the American Statistical Association, 53*(282), 457-481. https://doi.org/10.1080/01621459.1958.10501452

Kleinbaum, D. G., & Klein, M. (2012). *Survival analysis: A self-learning text* (3.ª ed.). Springer. https://doi.org/10.1007/978-1-4419-6646-9

Kohonen, T. (1990). The self-organizing map. *Proceedings of the IEEE, 78*(9), 1464-1480. https://doi.org/10.1109/5.58325

LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE, 86*(11), 2278-2324. https://doi.org/10.1109/5.726791

McInnes, L., Healy, J., & Melville, J. (2018). UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv*. https://arxiv.org/abs/1802.03426

Michalewicz, Z. (1996). *Genetic algorithms + data structures = evolution programs* (3.ª ed.). Springer. https://doi.org/10.1007/978-3-662-03315-9

Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. *Workshop track of the 1st International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/1301.3781

Montana, D. J., & Davis, L. (1989). Training feedforward neural networks using genetic algorithms. *Proceedings of the Eleventh International Joint Conference on Artificial Intelligence*, 762-767.

Pace, R. K., & Barry, R. (1997). Sparse spatial autoregressions. *Statistics & Probability Letters, 33*(3), 291-297. https://doi.org/10.1016/S0167-7152(96)00140-X

Pölsterl, S. (2020). scikit-survival: A library for time-to-event analysis built on top of scikit-learn. *Journal of Machine Learning Research, 21*(212), 1-6. http://jmlr.org/papers/v21/20-729.html

Rossi, P. H., Berk, R. A., & Lenihan, K. J. (1980). *Money, work, and crime: Experimental evidence*. Academic Press.

Schölkopf, B., Platt, J. C., Shawe-Taylor, J., Smola, A. J., & Williamson, R. C. (2001). Estimating the support of a high-dimensional distribution. *Neural Computation, 13*(7), 1443-1471. https://doi.org/10.1162/089976601750264965

Schölkopf, B., Smola, A., & Müller, K.-R. (1998). Nonlinear component analysis as a kernel eigenvalue problem. *Neural Computation, 10*(5), 1299-1319. https://doi.org/10.1162/089976698300017467

Schölkopf, B., Smola, A. J., Williamson, R. C., & Bartlett, P. L. (2000). New support vector algorithms. *Neural Computation, 12*(5), 1207-1245. https://doi.org/10.1162/089976600300015565

Scrucca, L. (2013). GA: A package for genetic algorithms in R. *Journal of Statistical Software, 53*(4), 1-37. https://doi.org/10.18637/jss.v053.i04

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research, 15*(56), 1929-1958. http://jmlr.org/papers/v15/srivastava14a.html

Strobl, C., Boulesteix, A.-L., Zeileis, A., & Hothorn, T. (2007). Bias in random forest variable importance measures: Illustrations, sources and a solution. *BMC Bioinformatics, 8*, 25. https://doi.org/10.1186/1471-2105-8-25

Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to sequence learning with neural networks. *Advances in Neural Information Processing Systems, 27*, 3104-3112.

Therneau, T. M., & Grambsch, P. M. (2000). *Modeling survival data: Extending the Cox model*. Springer. https://doi.org/10.1007/978-1-4757-3294-8

van der Maaten, L. (2014). Accelerating t-SNE using tree-based algorithms. *Journal of Machine Learning Research, 15*, 3221-3245.

van der Maaten, L., & Hinton, G. (2008). Visualizing data using t-SNE. *Journal of Machine Learning Research, 9*, 2579-2605.

Vapnik, V. N. (1995). *The nature of statistical learning theory*. Springer.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems, 30*, 5998-6008.

Volpacchio, M. (s.f.). *Data Mining Avanzado: presentaciones, apuntes y notebooks de clase* [Material de cátedra]. Universidad Austral, Maestría en Explotación de Datos y Gestión del Conocimiento.

Wolberg, W., Mangasarian, O., Street, N., & Street, W. (1993). *Breast Cancer Wisconsin (Diagnostic)* [Conjunto de datos]. UCI Machine Learning Repository. https://doi.org/10.24432/C5DW2B